

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống quản lý chung cư

VŨ THU TRANG

trang.vt225232@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Nguyễn Thị Thanh Nga

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống quản lý chung cư

VŨ THU TRANG

trang.vt225232@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Nguyễn Thị Thanh Nga

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến cô Nguyễn Thị Thanh Nga. Sự chỉ bảo tận tâm và những góp ý quý báu của cô trong suốt quá trình thực hiện đồ án tốt nghiệp đã giúp em mở rộng tư duy, trau dồi kiến thức chuyên môn để hoàn thiện đồ án một cách tốt nhất. Em cũng xin cảm ơn các thầy cô giáo đã tận tụy dạy dỗ em suốt bốn năm qua, trang bị cho em những hành trang quý giá cho tương lai.

Em xin cảm ơn gia đình đã luôn là điểm tựa vững chắc, luôn sẻ chia, động viên và tạo mọi điều kiện tốt nhất để em yên tâm học tập. Tiếp đến, em xin gửi lời cảm ơn đến những người bạn thân thiết đã luôn đồng hành, cùng nhau học tập, giúp đỡ và chia sẻ mọi áp lực trong suốt những năm tháng đại học.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh đô thị hóa phát triển, việc vận hành các tổ hợp chung cư gồm nhiều tòa nhà cao tầng đang đối mặt trước thách thức về quản lý và vận hành với lượng lớn căn hộ và cư dân. Hiện tại, các ban quản lý thường giải quyết bằng phương thức thủ công hoặc dùng công cụ hỗ trợ cơ bản như Excel. Tuy nhiên, hướng tiếp cận này bộc lộ rõ hạn chế khi quy mô mở rộng: hiệu quả quản lý chưa cao, dữ liệu bị phân mảnh, thiếu đồng bộ và gây khó khăn trong việc kết nối giữa ban quản trị với cư dân.

Đồ án lựa chọn hướng tiếp cận xây dựng một hệ thống quản lý tích hợp đa nền tảng. Hướng đi này giúp liên kết dữ liệu của toàn bộ các tòa nhà trong tổ hợp chung cư về một hệ thống tập trung, vừa đảm bảo tính quản trị của ban quản trị, vừa duy trì tính linh hoạt cho quản lý tại từng tòa nhà.

Tổng quan giải pháp bao gồm một trang web cung cấp giao diện thuận tiện cho việc quản trị của ban quản lý, đồng thời web cũng cho phép cư dân đăng nhập. Song song với đó là ứng dụng di động dành riêng cho cư dân nhằm tối ưu hóa sự tiện lợi.

Đồ án đã chuẩn hóa và số hóa toàn diện quy trình vận hành chuỗi tòa nhà phức tạp thành một mô hình quản lý thông tin thông suốt, có khả năng mở rộng quy mô linh hoạt. Kết quả đạt được sau cùng là một hệ thống đa nền tảng hoàn chỉnh, hoạt động ổn định và đồng bộ, giúp tối ưu hóa năng suất quản lý cho ban quản trị và nâng cao chất lượng trải nghiệm sống cho cư dân trong toàn bộ hệ thống chung cư.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã Đăng nhập	8
2.2.3 Biểu đồ use case phân rã Quản lý khoản phí	9
2.2.4 Biểu đồ use case phân rã Quản lý căn hộ	10
2.2.5 Biểu đồ use case phân rã Quản lý cư dân	10
2.2.6 Biểu đồ use case phân rã Quản lý phương tiện	11
2.2.7 Biểu đồ use case phân rã Quản lý thông báo	12
2.2.8 Biểu đồ use case phân rã Quản lý kiến nghị	12
2.2.9 Biểu đồ use case phân rã Quản lý tài khoản	13
2.2.10 Biểu đồ use case phân rã Quản lý tài sản và tài liệu	14
2.2.11 Biểu đồ use case phân rã Quản lý tiện ích	14
2.2.12 Quy trình nghiệp vụ.....	15
2.3 Đặc tả chức năng	16
2.3.1 Đặc tả use case Lấy lại mật khẩu	16
2.3.2 Đặc tả use case Thanh toán khoản phí.....	17
2.3.3 Đặc tả use case Tạo bảng thu phí	18

2.3.4 Đặc tả use case Thêm mới tài khoản căn hộ	19
2.3.5 Đặc tả use case Vô hiệu hóa tài khoản căn hộ.....	20
2.4 Yêu cầu phi chức năng	21
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	23
3.1 Tổng quan kiến trúc hệ thống	23
3.2 Nền tảng Backend và hệ quản trị cơ sở dữ liệu.....	24
3.2.1 Nền tảng Backend	24
3.2.2 Hệ quản trị cơ sở dữ liệu	25
3.3 Nền tảng Frontend	25
3.3.1 Web	25
3.3.2 Ứng dụng di động	26
3.4 Cơ chế bộ nhớ đệm và tối ưu hóa hiệu năng	27
3.5 Giải pháp an ninh hệ thống và bảo mật dữ liệu	29
3.6 Các giải pháp hỗ trợ.....	30
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	32
4.1 Thiết kế kiến trúc.....	32
4.1.1 Lựa chọn kiến trúc phần mềm	32
4.1.2 Thiết kế tổng quan.....	33
4.1.3 Thiết kế chi tiết gói	35
4.2 Thiết kế chi tiết.....	37
4.2.1 Thiết kế giao diện	37
4.2.2 Thiết kế lớp	40
4.2.3 Thiết kế cơ sở dữ liệu	42
4.3 Xây dựng ứng dụng.....	45
4.3.1 Thư viện và công cụ sử dụng.....	45

4.3.2 Kết quả đạt được	45
4.3.3 Minh họa các chức năng chính	46
4.4 Kiểm thử.....	49
4.4.1 Phương pháp và kỹ thuật kiểm thử áp dụng.....	49
4.4.2 Kịch bản kiểm thử chức năng Tạo bảng thu phí tự động	49
4.4.3 Kịch bản kiểm thử chức năng thanh toán hóa đơn.....	50
4.4.4 Tổng kết và phân tích kết quả kiểm thử.....	51
4.5 Triển khai	51
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	54
5.1 Kiến trúc xác thực lai đa nền tảng và mô hình kiểm soát quyền hạn dynamic RBAC.....	54
5.1.1 Vấn đề.....	54
5.1.2 Giải pháp	54
5.1.3 Kết quả	55
5.2 Giải pháp thanh toán tự động với Webhook SePay	56
5.2.1 Vấn đề.....	56
5.2.2 Giải pháp	56
5.2.3 Kết quả	58
5.3 Giải pháp tối ưu hóa hiệu năng với Celery và Redis Caching	58
5.3.1 Vấn đề.....	58
5.3.2 Giải pháp	59
5.3.3 Kết quả	61
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	62
6.1 Kết luận.....	62
6.2 Hướng phát triển.....	62
TÀI LIỆU THAM KHẢO.....	64

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	7
Hình 2.2	Biểu đồ use case phân rã Đăng nhập	8
Hình 2.3	Biểu đồ use case phân rã Quản lý khoản phí	9
Hình 2.4	Biểu đồ use case phân rã Quản lý căn hộ	10
Hình 2.5	Biểu đồ use case phân rã Quản lý cư dân	10
Hình 2.6	Biểu đồ use case phân rã Quản lý phương tiện	11
Hình 2.7	Biểu đồ use case phân rã Quản lý thông báo	12
Hình 2.8	Biểu đồ use case phân rã Quản lý kiến nghị	12
Hình 2.9	Biểu đồ use case phân rã Quản lý tài khoản	13
Hình 2.10	Biểu đồ use case phân rã Quản lý tài sản và tài liệu	14
Hình 2.11	Biểu đồ use case phân rã Quản lý tiện ích	14
Hình 2.12	Biểu đồ hoạt động nghiệp vụ tạo khoản phí và thanh toán phí .	16
Hình 4.1	Biểu đồ phụ thuộc gói tổng quan	34
Hình 4.2	Thiết kế chi tiết gói cho quản lý khoản phí	36
Hình 4.3	Thiết kế giao diện cho màn hình tạo bảng thu phí trên web . .	38
Hình 4.4	Thiết kế giao diện cho màn hình quản lý căn hộ trên web . . .	38
Hình 4.5	Thiết kế giao diện cho màn hình thanh toán trên web	39
Hình 4.6	Thiết kế giao diện cho màn hình khoản phí trên app mobile . .	39
Hình 4.7	Biểu đồ trình tự cho usecase Tạo bảng thu phí	41
Hình 4.8	Biểu đồ trình tự cho usecase Thanh toán	42
Hình 4.9	Biểu đồ thực thể liên kết	43
Hình 4.10	Màn hình quản lý khoản phí cho ban quản trị	47
Hình 4.11	Màn hình thống kê khoản phí cho ban quản trị	47
Hình 4.12	Màn hình khoản phí cho cư dân	48
Hình 4.13	Màn hình thông tin về căn hộ cho cư dân	48

DANH MỤC BẢNG BIỂU

Bảng 2.1	Bảng so sánh và đánh giá định vị giải pháp của đề tài	6
Bảng 4.1	Danh sách thư viện và công cụ sử dụng	45
Bảng 4.2	Thống kê quy mô mã nguồn hệ thống	46
Bảng 4.3	Kịch bản kiểm thử chi tiết chức năng tạo bảng thu phí	50
Bảng 4.4	Kịch bản kiểm thử chi tiết chức năng Thanh toán hóa đơn . . .	51
Bảng 4.5	Hạ tầng triển khai các thành phần hệ thống	52

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng, phương thức kết nối giữa các thành phần phần mềm (Application Programming Interface).
CORS	Cơ chế cho phép tài nguyên hạn chế trên một trang web được yêu cầu từ một tên miền khác (Cross-Origin Resource Sharing).
CSS	Ngôn ngữ định kiểu biểu mẫu dùng để miêu tả cách trình bày của văn bản HTML (Cascading Style Sheets).
JWT	Chuỗi mã hóa dạng JSON dùng để xác thực quyền truy cập giữa các hệ thống (JSON Web Token).
ORM	Kỹ thuật lập trình ánh xạ cơ sở dữ liệu quan hệ sang các đối tượng trong ngôn ngữ lập trình (Object-Relational Mapping).
SCSS	Cú pháp mở rộng nâng cao của CSS, hỗ trợ biến, lồng cú pháp và hàm (Sassy Cascading Style Sheets).
SMTP	Giao thức chuẩn mã nguồn truyền tải thư điện tử qua mạng Internet (Simple Mail Transfer Protocol).
URL	Chuỗi ký tự chuẩn hóa dùng để định vị một tài nguyên trên mạng Internet (Uniform Resource Locator).
Web	Không gian thông tin toàn cầu nơi các tài liệu và tài nguyên mạng được định danh bằng URL (World Wide Web).
XSS	Lỗ hổng bảo mật cho phép kẻ tấn công chèn các đoạn mã kịch bản độc hại vào trang web (Cross-Site Scripting).
RBAC	Mô hình kiểm soát truy cập dựa trên vai trò (Role-Based Access Control)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Hiện nay, các tổ hợp chung cư quy mô lớn bao gồm nhiều tòa nhà cao tầng đang trở thành mô hình nhà ở phổ biến tại các đô thị hiện đại. Để đảm bảo toàn bộ hệ thống này vận hành ổn định, an toàn và duy trì chất lượng không gian sống, công tác quản lý vận hành đòi hỏi phải được thực hiện một cách liên tục, định kỳ và chính xác.

Với mô hình quản lý đa tòa nhà, Ban quản trị phải kiểm soát một khối lượng thông tin phức tạp. Từ việc quản lý thông tin biến động cư dân (lịch sử cư trú, tạm trú, tạm vắng), quản lý phương tiện giao thông, cho đến việc tiếp nhận và xử lý dòng phản ánh, kiến nghị liên tục từ hàng nghìn cư dân. Với phương thức quản lý thủ công, quy trình tổng hợp, tính toán và thông báo các danh mục phí hàng tháng vẫn phụ thuộc vào phương thức thủ công và các công cụ bảng tính như Excel. Khi quy mô dự án mở rộng ra nhiều tòa nhà, cách làm này bộc lộ những hạn chế: dễ xảy ra sai sót trong số liệu tài chính, tốc độ truyền tải thông tin đến cư dân bị chậm trễ, quy trình tương tác giữa Ban quản trị và người dân bị nghẽn mạch, gây mất thời gian cho cả hai bên.

Từ thực trạng trên, bài toán đặt ra là phải tự động hóa quy trình quản lý tài chính và vận hành, đồng thời số hóa kênh tương tác giữa đơn vị quản lý và cộng đồng cư dân trong một mô hình đa tòa nhà tập trung. Nếu bài toán này được giải quyết, lợi ích mang lại là rất lớn. Ban quản trị sẽ được giảm tải áp lực công việc, tối ưu hóa quy trình thu phí, hạn chế thất thoát tài chính và nâng cao tính minh bạch, chuyên nghiệp trong quản lý. Cư dân có thể tiếp cận thông tin nhanh chóng, thực hiện nghĩa vụ tài chính tiện lợi, và dễ dàng phản hồi ý kiến, từ đó nâng cao chất lượng trải nghiệm sống.

Mô hình giải quyết bài toán có thể triển khai rộng rãi cho bất kỳ tổ hợp chung cư, khu đô thị phức hợp hoặc các khu quản lý ký túc xá có quy mô và cấu trúc tương tự trong xu hướng phát triển đô thị thông minh hiện nay.

1.2 Mục tiêu và phạm vi đề tài

Trên thực tế, phương thức quản lý thủ công bộc lộ nhiều hạn chế khi quy mô dân cư tăng cao. Quy trình tính toán và thu các loại phí hàng tháng dễ xảy ra sai sót hoặc chậm trễ. Việc quản lý thiếu tính đồng bộ, chưa được tự động hóa. Hệ thống vận hành chưa đảm bảo được hiệu năng và tính an toàn bảo mật dữ liệu khi phát sinh lượng truy cập lớn. Mặc dù trên thị trường đã xuất hiện một số hệ thống hỗ

trợ quản lý, nhưng khi quy mô dự án mở rộng, các hệ thống này vẫn bộc lộ nhiều hạn chế. Việc dữ liệu bị phân mảnh, thiếu tính đồng bộ trung tâm dễ dẫn đến sai sót trong sổ liệu tài chính, tốc độ truyền tải thông tin đến cư dân bị chậm trễ và quy trình tương tác hai chiều bị nghẽn mạch. Thực trạng này không chỉ gây mất thời gian, tốn kém nguồn lực cho Ban quản lý mà còn làm giảm chất lượng trải nghiệm sống của cư dân. Do đó, việc xây dựng một giải pháp công nghệ tích hợp đa nền tảng mang tính đồng bộ và tập trung là nhu cầu cấp thiết.

Từ những phân tích và đánh giá trên, đề tài hướng tới mục tiêu giải quyết bài toán tối ưu hóa vận hành đa tòa nhà bằng cách xây dựng một hệ thống phần mềm quản lý tích hợp đa nền tảng mang tính đồng bộ và tập trung cao. Hệ thống được phát triển tập trung vào nhóm chức năng lõi nhằm số hóa toàn diện từ quy trình xử lý tài chính hóa đơn, quản trị hạ tầng căn hộ, nhân khẩu xuyên suốt, điều phối các hạng mục vận hành nội khu, cho đến việc thiết lập kênh tương tác đa chiều có tích hợp trí tuệ nhân tạo hỗ trợ giải đáp tự động cho cư dân. Điểm đột phá của phần mềm nằm ở việc giải quyết bài toán kỹ thuật thông qua cơ chế tối ưu hóa bộ nhớ đệm giúp nâng cao hiệu năng hệ thống, kiến trúc bảo mật đa tầng chống các lỗ hổng tấn công mạng và quy trình tự động hóa giám sát an toàn bằng nhật ký hoạt động. Giải pháp công nghệ này không chỉ tiết kiệm nguồn lực vận hành mà còn mang lại một hệ sinh thái số an toàn, bảo mật và thông suốt cho cả ban quản lý và cư dân.

1.3 Định hướng giải pháp

Để giải quyết bài toán, đề án phát triển hệ thống theo kiến trúc tách biệt giữa Backend và Frontend với các công nghệ:

Backend: Sử dụng Django Rest Framework kết hợp cơ sở dữ liệu PostgreSQL với khả năng bảo mật cao, hỗ trợ xây dựng hệ thống API mạnh mẽ, quản lý các mối quan hệ dữ liệu phức tạp của mô hình nhiều tòa nhà tốt và tối ưu hóa thời gian phát triển.

Frontend: Sử dụng React cho Web và React Native (TypeScript) cho ứng dụng di động. Lựa chọn này giúp tối ưu hóa giao diện, đồng bộ trải nghiệm người dùng và tái sử dụng mã nguồn hiệu quả giữa các nền tảng di động.

Tối ưu và bảo mật: Tích hợp Redis đóng vai trò bộ nhớ đệm (Cache data), giúp tăng tốc độ phản hồi hệ thống và tối ưu hóa hiệu năng khi truy xuất dữ liệu lớn. Sử dụng cơ chế mã hóa JWT để quản lý phiên đăng nhập, xác thực người dùng và phân quyền an toàn giữa các nhóm đối tượng trên cả hai nền tảng Web và Mobile.

Hệ thống cung cấp một nền tảng quản trị tập trung giúp ban quản lý điều hành,

lập bảng phí và phê duyệt yêu cầu trên giao diện Web. Song song đó, cư dân sẽ nhận thông báo, theo dõi hóa đơn, thanh toán và gửi kiến nghị thời gian thực thông qua ứng dụng di động và cả web. Hệ thống đã tự động hóa một số quy trình và quản lý nhân khẩu cho mô hình đa tòa nhà; số hóa kênh tương tác hai chiều và xử lý bất đồng bộ các tác vụ vận hành giữa ban quản lý và cư dân.

Kết quả đạt được là một hệ thống phần mềm hoàn thiện gồm trang Web quản trị và ứng dụng di động vận hành ổn định, đáp ứng tốt yêu cầu về hiệu năng, bảo mật.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức thành các chương chính nhằm trình bày một cách hệ thống từ khâu khảo sát bài toán, phân tích thiết kế đến hiện thực hóa giải pháp và đánh giá kết quả đạt được.

Chương 2 tập trung vào việc khảo sát hiện trạng và phân tích thiết kế hệ thống quản lý tổ hợp chung cư đa tòa nhà. Tại đây, đồ án đi sâu vào việc xác định các tác nhân tham gia vào hệ thống. Nội dung chương bao gồm việc xây dựng biểu đồ ca sử dụng tổng quát và các biểu đồ ca sử dụng phân rã các nhóm chức năng chính, đặc tả chi tiết một số ca sử dụng quan trọng và yêu cầu phi chức năng về hiệu năng, an toàn bảo mật.

Chương 3 trình bày về cơ sở lý thuyết và các công nghệ nền tảng tạo tiền đề cho giải pháp đề xuất. Nội dung chương tập trung giới thiệu về kiến trúc hệ thống, cơ chế hoạt động của Web và ứng dụng di động trong việc đồng bộ dữ liệu. Bên cạnh đó, các công nghệ lõi như framework phát triển phía Backend và Frontend, hệ quản trị cơ sở dữ liệu, giải pháp bộ nhớ đệm tối ưu hiệu năng và nền tảng trí tuệ nhân tạo phục vụ xây dựng Chatbot hỗ trợ cư dân cũng được phân tích làm rõ, khẳng định sự phù hợp cho bài toán hệ thống lớn.

Chương 4 đi vào quy trình thiết kế kiến trúc và hiện thực hóa chi tiết hệ thống phần mềm. Đầu tiên, chương này mô tả kiến trúc phần mềm sử dụng mô hình MVC, đồng thời mô hình hóa sự phụ thuộc giữa các thành phần thông qua biểu đồ gói tổng quan và thiết kế chi tiết gói nghiệp vụ. Tiếp theo, thiết kế chi tiết được triển khai nhằm chuẩn hóa quy tắc giao diện đa nền tảng, thiết kế thuộc tính phương thức của các lớp chủ đạo kết hợp biểu đồ trình tự cho các ca sử dụng quan trọng, và cấu hình sơ đồ thực thể liên kết cho cơ sở dữ liệu quan hệ. Trong phần xây dựng ứng dụng, chương này tổng hợp các thư viện, công cụ lập trình cốt lõi và trực quan hóa giao diện các chức năng chính. Cuối cùng, quy trình kiểm thử hộp đen được sử dụng để đánh giá các trường hợp kiểm thử nghiêm ngặt cùng giải pháp triển khai thực tế.

Chương 5 là chương đóng góp giải pháp cốt lõi, nơi trình bày những kết quả

ngiên cứu thực nghiệm và các giải pháp kỹ thuật tâm đắc nhất trong suốt quá trình thực hiện đồ án tốt nghiệp. Đầu tiên, chương này đi sâu phân tích giải pháp tối ưu hóa hiệu năng chịu tải và bảo mật cho tổ hợp đa tòa nhà dựa trên sự kết hợp giữa kiến trúc phân tầng, cơ chế bộ nhớ đệm đa tầng Cache-Aside với Redis và Middleware tự động ghi nhật ký hệ thống, đi kèm các số liệu trực quan hóa từ kịch bản kiểm thử tải lớn. Tiếp theo, giải pháp xây dựng và tích hợp trợ lý ảo Chatbot trí tuệ nhân tạo hỗ trợ cư dân trực tuyến cũng được làm rõ thông qua quy trình chuẩn hóa kho tri thức nội bộ kết hợp xử lý ngôn ngữ tự nhiên, đồng thời chứng minh hiệu quả thực tế thông qua các chỉ số về độ chính xác nhận diện ý định và tốc độ phản hồi thời gian thực của hệ thống.

Chương 6 tổng kết toàn bộ quá trình thực hiện đồ án tốt nghiệp. Chương này chỉ ra các ưu điểm về tính đồng bộ dữ liệu, năng lực bảo mật và hiệu năng vận hành thực tế. Đồng thời, những hạn chế kỹ thuật còn tồn tại cũng được phân tích cụ thể, làm cơ sở đề xuất các hướng nghiên cứu mở rộng và phát triển tính năng thông minh cho sản phẩm trong giai đoạn tiếp theo.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã khái quát bối cảnh phát triển, lý do hình thành đề tài, cũng như xác định rõ mục tiêu xây dựng một giải pháp công nghệ quản lý tổ hợp chung cư đa tòa nhà tích hợp đa nền tảng. Để hiện thực hóa mục tiêu đó, Chương 2 đóng vai trò nền tảng khi đi sâu vào việc khảo sát hiện trạng bài toán vận hành thực tế và tiến hành phân tích yêu cầu chức năng và yêu cầu phi chức năng của hệ thống một cách toàn diện.

2.1 Khảo sát hiện trạng

Quá trình phân tích hiện trạng và định hình các yêu cầu phần mềm cho đề tài được thực hiện dựa trên việc tổng hợp ba nguồn thông tin bao gồm: các yêu cầu nghiệp vụ cốt lõi từ bài toán quản lý vận hành thực tế, cơ chế xử lý dữ liệu của các công cụ bảng tính thủ công, và các giải pháp phần mềm thương mại hiện hành.

Qua khảo sát, tìm hiểu nghiệp vụ vận hành thực tế tại các tổ hợp chung cư, một hệ thống tối ưu đòi hỏi phải có khả năng bao quát toàn bộ hoạt động vận hành hằng ngày nhưng phải đơn giản hóa quy trình điều hành chuỗi đa tòa nhà. Về phía cư dân, yêu cầu chính là sự minh bạch trong việc tra cứu các khoản chi phí, tốc độ tiếp nhận phản hồi nhanh chóng và tính tiện lợi khi đăng ký các dịch vụ hành chính hay tiện ích nội khu. Nếu xét về các công cụ hiện có, phần lớn các tòa nhà vẫn duy trì việc quản lý nhân khẩu và tính toán hóa đơn bằng các bảng tính Excel độc lập. Phương thức này bộc lộ nhược điểm chí tử khi quy mô dân cư tăng cao: dữ liệu phân mảnh, dễ xảy ra sai sót trong công thức tính toán tài chính và hoàn toàn không có kênh tương tác thời gian thực với người dân.

Đối với nguồn khảo sát từ các ứng dụng tương tự trên thị trường, đề tài tiến hành phân tích hai giải pháp tiêu biểu đang được áp dụng tại nhiều dự án hiện nay, bao gồm:

- **Phần mềm HomeID:** Giải pháp này hỗ trợ số hóa thông tin căn hộ và tự động lập hóa đơn dịch vụ định kỳ. Tuy nhiên, hệ thống định hướng tích hợp sâu sàn thương mại điện tử trực tuyến (HomeID Market) để kết nối các nhà cung cấp nhu yếu phẩm ngoại khu vào ứng dụng của cư dân. Việc phân phối không gian cho các tính năng mua sắm làm giảm tính chuyên sâu đối với các nghiệp vụ quản trị hành chính của Ban quản lý.
- **Phần mềm Building Care:** Là một nền tảng chuyển đổi số chuyên sâu trong quản lý vận hành tòa nhà với các phân hệ phân cấp rất chi tiết. Dù cung cấp bộ tính năng mạnh mẽ, nhưng việc triển khai thương mại của hệ thống này được

chia theo các gói dịch vụ giới hạn dựa trên quy mô căn hộ, dẫn đến chi phí duy trì đắt đỏ theo tháng, đồng thời không cho phép Ban quản lý tự chủ về hạ tầng lưu trữ dữ liệu nội bộ.

Nhằm làm rõ tính cấp thiết và lý do phát triển của đề tài trước các giải pháp thương mại sẵn có, Bảng 2.1 tiến hành phân tích, so sánh chi tiết các đặc tính kỹ thuật và mô hình triển khai giữa các hướng tiếp cận.

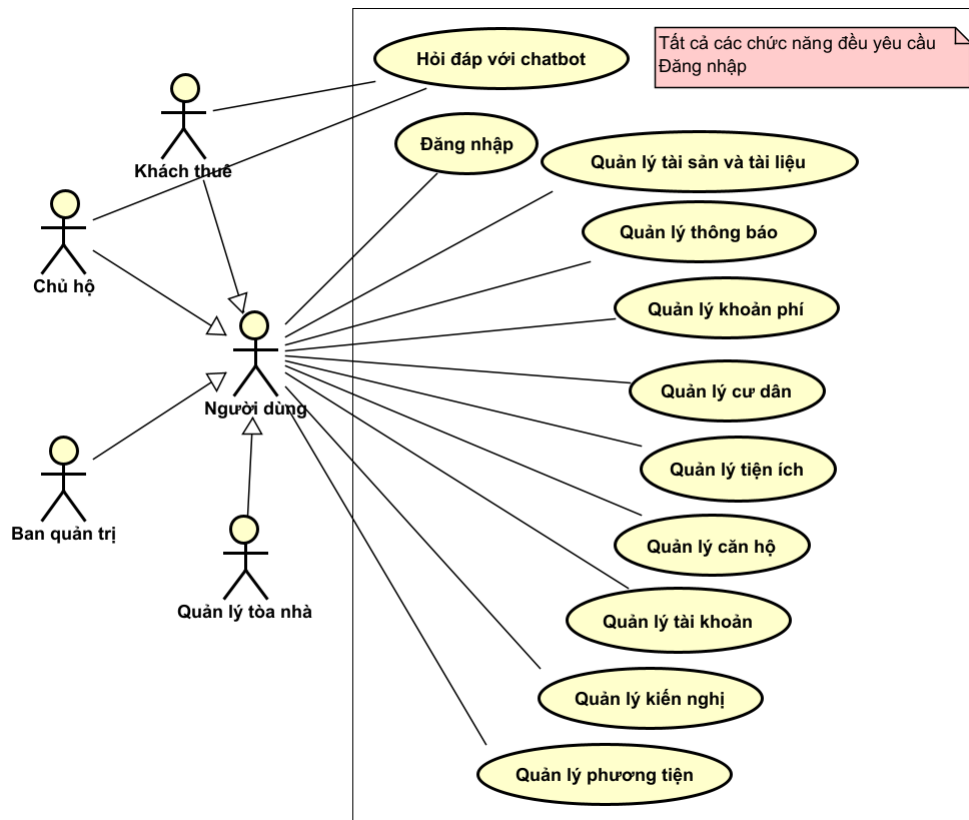
Tiêu chí	Excel	Phần mềm thương mại	Đề xuất
Quản trị chuỗi đa tòa nhà	Thấp	Tốt	Tốt
Tính chủ quyền dữ liệu	Thấp	Thấp	Tuyệt đối
Tính đa nền tảng	Hạn chế	Tốt	Tốt
Hỗ trợ tương tác thông minh	Không có	Trung bình	Tốt
Chi phí vận hành	Thấp	Cao	Tối ưu

Bảng 2.1: Bảng so sánh và đánh giá định vị giải pháp của đề tài

Từ kết quả phân tích và đánh giá so sánh ở trên, có thể thấy các giải pháp thương mại dù đã rất hoàn thiện về mặt tính năng tổng quát nhưng vẫn tồn tại một số đối với các nghiệp vụ đặc thù cần tính tùy biến cao, chi phí vận hành chuỗi dài hạn và đặc biệt là vấn đề tự chủ an toàn dữ liệu nội bộ. Do đó, mục tiêu của đề tài không nhằm mục đích thương mại, mà tập trung xây dựng một giải pháp chuyên biệt, tối ưu hiệu năng chịu tải đa tầng dựa trên bộ nhớ đệm Redis, bảo mật dữ liệu tuyệt đối và tạo đột phá trong tương tác cư dân thông qua trợ lý ảo Chatbot thông minh dựa trên xử lý ngôn ngữ tự nhiên.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát

Hệ thống quản lý chung cư bao gồm các tác nhân sau:

Người dùng: Là tác nhân tổng quát hóa đại diện cho tất cả các đối tượng có tài khoản và tương tác với hệ thống.

Ban quản trị: Có quyền quản lý toàn bộ các tòa nhà, cấp phát tài khoản quản trị cho các quản lý tòa nhà, đồng thời giám sát dữ liệu tổng thể.

Quản lý tòa nhà: Được phân quyền quản lý trực tiếp một tòa nhà cụ thể. Có vai trò điều hành các nghiệp vụ quản lý trong phạm vi tòa nhà được phân công.

Chủ hộ: Là cư dân sở hữu hợp pháp căn hộ. Có vai trò quản lý thông tin căn hộ của mình, quản lý thông tin khách thuê (nếu có), thực hiện các nghĩa vụ của căn hộ như nộp phí hàng tháng,... trừ căn hộ đã cho thuê thì chỉ có thể xem thông tin.

Khách thuê: Là đối tượng thuê lại căn hộ từ chủ hộ, có các quyền và nghĩa vụ đối với căn hộ đang thuê tương đương chủ hộ.

Hệ thống bao gồm các nhóm chức năng cốt lõi sau:

Đăng nhập: Use case bắt buộc đối với tất cả chức năng, đảm bảo xác thực danh tính và điều hướng giao diện phù hợp với quyền của từng tài khoản.

Quản lý khoản phí: Hỗ trợ tính toán, tự động lập bảng kê các loại phí định kỳ, theo dõi trạng thái thanh toán và tổng hợp báo cáo doanh thu tài chính.

Quản lý căn hộ: Lưu trữ và quản lý thông tin chi tiết về hạ tầng các căn hộ thuộc nhiều tòa nhà khác nhau trong tổ hợp.

Quản lý cư dân: Theo dõi danh sách nhân khẩu, cập nhật biến động dân cư.

Quản lý phương tiện: Quản lý các loại phương tiện giao thông của cư dân đăng ký gửi tại chung cư.

Quản lý thông báo: Hỗ trợ ban quản lý biên soạn, gửi các thông tin nhắc phí hoặc sự kiện tòa nhà tới từng nhóm cư dân đích.

Quản lý kiến nghị: Tiếp nhận các phản ánh, đóng góp ý kiến từ cư dân; cập nhật và công khai tiến độ xử lý của ban quản lý.

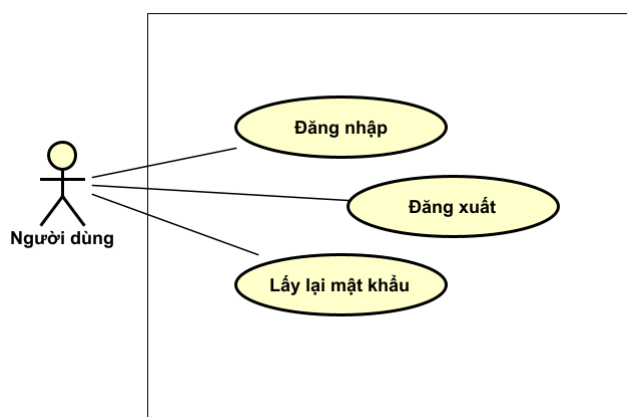
Quản lý tài sản và tài liệu: Hệ thống hóa danh mục tài sản công, lịch bảo dưỡng cơ sở vật chất và lưu trữ các văn bản quy định, tài liệu hướng dẫn của tòa nhà.

Quản lý tiện ích: Điều phối việc đăng ký và sử dụng các không gian, dịch vụ tiện ích chung.

Quản lý tài khoản: Cho phép người dùng chỉnh sửa thông tin cá nhân, cập nhật mật khẩu hoặc vô hiệu hóa tài khoản tùy theo cấp độ phân quyền.

Hỏi đáp với chatbot: Cung cấp kênh tương tác AI hỗ trợ giải đáp nhanh các thắc mắc của cư dân về căn hộ, quy định và dịch vụ tại chung cư.

2.2.2 Biểu đồ use case phân rã Đăng nhập



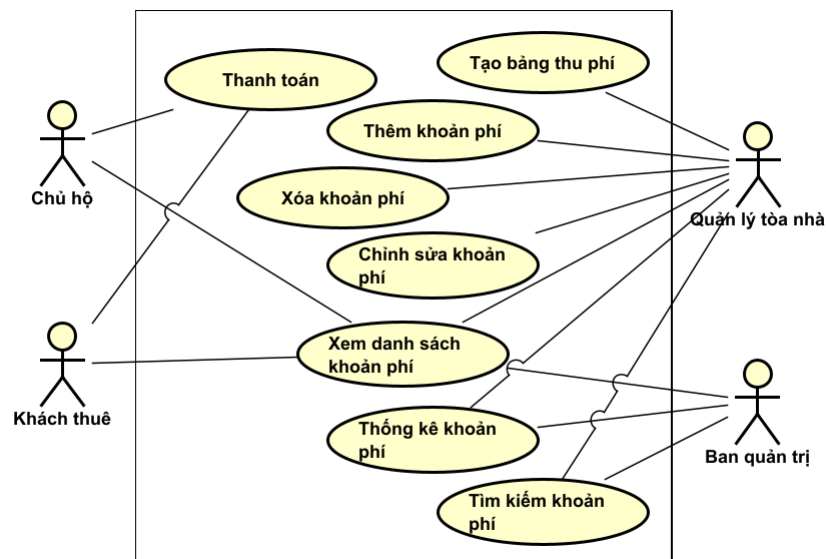
Hình 2.2: Biểu đồ use case phân rã Đăng nhập

Đăng nhập: Cho phép mọi người dùng cung cấp tài khoản để truy cập vào hệ thống theo đúng vai trò được phân quyền.

Đăng xuất: Kết thúc phiên làm việc hiện tại của người dùng.

Lấy lại mật khẩu: Hỗ trợ người dùng thiết lập lại mật khẩu truy cập trong trường hợp quên mật khẩu đăng nhập.

2.2.3 Biểu đồ use case phân rã Quản lý khoản phí



Hình 2.3: Biểu đồ use case phân rã Quản lý khoản phí

Xem danh sách khoản phí: Cho phép ban quản trị, quản lý tòa nhà tra cứu, theo dõi chi tiết toàn bộ khoản phí của các căn hộ, cư dân chỉ có thể xem thông tin của căn hộ mình.

Thanh toán: Hỗ trợ cư dân thực hiện quy trình nộp phí trực tuyến qua hệ thống.

Tìm kiếm khoản phí: Tra cứu khoản phí theo tên khoản phí, theo căn hộ hoặc loại khoản phí cụ thể.

Thống kê khoản phí: Hỗ trợ ban quản trị, quản lý tổng hợp dữ liệu và xuất báo cáo về việc đóng phí định kỳ theo tháng, năm.

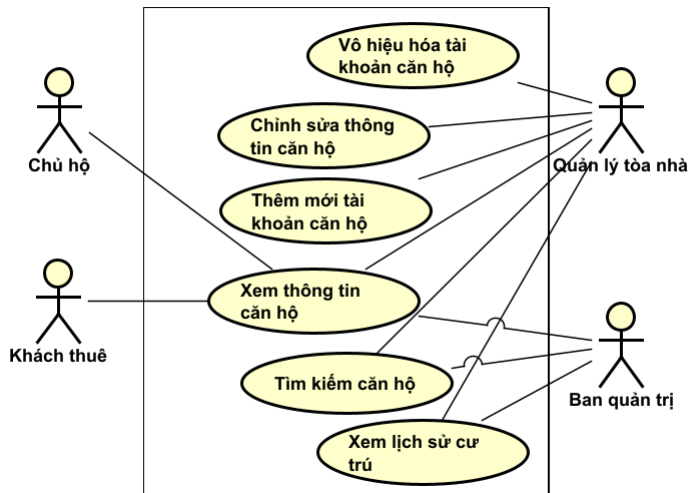
Tạo bảng thu phí: Khởi tạo kỳ bảng thu phí tháng mới cho các căn hộ.

Thêm khoản phí: Hỗ trợ quản lý tòa nhà tạo thêm một loại khoản phí mới vào hệ thống.

Chỉnh sửa khoản phí: Cập nhật, thay đổi thông tin của một khoản phí.

Xóa khoản phí: Xóa một khoản phí khỏi hệ thống.

2.2.4 Biểu đồ use case phân rã Quản lý căn hộ



Hình 2.4: Biểu đồ use case phân rã Quản lý căn hộ

Vô hiệu hóa tài khoản căn hộ: Cho phép Quản lý tòa nhà khóa quyền truy cập của tài khoản liên kết với căn hộ.

Chính sửa thông tin căn hộ: Cập nhật các thay đổi thông tin cơ bản của căn hộ.

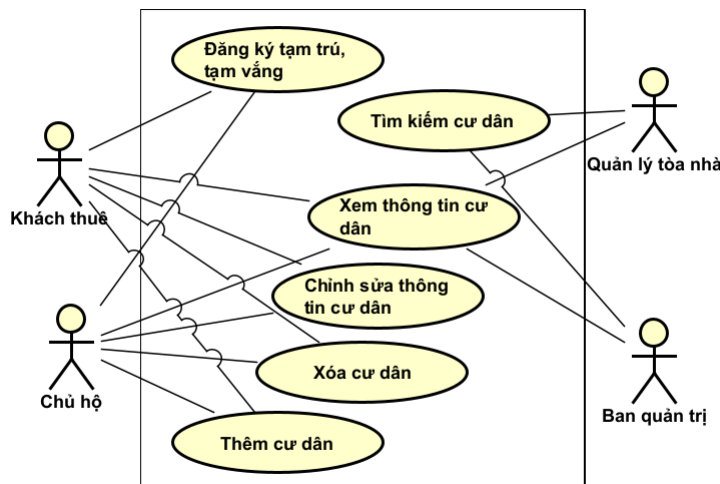
Thêm mới tài khoản căn hộ: Cho phép Quản lý tòa nhà cấp mới và khởi tạo tài khoản đăng nhập cho căn hộ.

Xem thông tin căn hộ: Cho phép ban quản trị, quản lý tòa nhà tra cứu, theo dõi thông tin chi tiết toàn bộ căn hộ, cư dân chỉ có thể xem thông tin của căn hộ mình.

Tìm kiếm căn hộ: Hỗ trợ tra cứu nhanh căn hộ theo mã căn hộ, email chủ hộ.

Xem lịch sử cư trú: Cho phép Ban quản trị theo dõi lịch sử cư trú của các căn hộ.

2.2.5 Biểu đồ use case phân rã Quản lý cư dân



Hình 2.5: Biểu đồ use case phân rã Quản lý cư dân

Đăng ký tạm trú, tạm vắng: Cư dân khai báo trực tuyến các thông tin thay đổi về trạng thái cư trú.

Tìm kiếm cư dân: Cho phép quản lý tòa nhà và Ban quản trị tra cứu thông tin của một người dân cụ thể trong hệ thống.

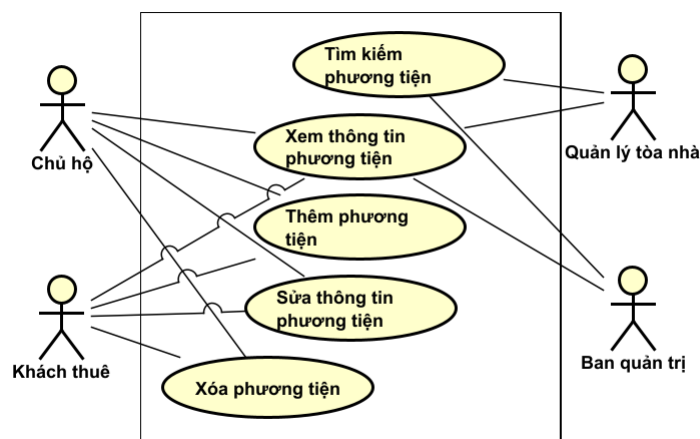
Xem thông tin cư dân: Cho phép ban quản trị, quản lý tòa nhà tra cứu, theo dõi thông tin chi tiết toàn bộ cư dân, cư dân chỉ có thể xem thông tin của căn hộ mình.

Chỉnh sửa thông tin cư dân: Cập nhật, sửa đổi các thông tin của cư dân.

Xóa cư dân: Hỗ trợ Chủ hộ và Quản lý tòa nhà loại bỏ thông tin của một nhân khẩu ra khỏi danh sách quản lý khi họ chuyển đi.

Thêm cư dân: Thêm thành viên mới vào căn hộ.

2.2.6 Biểu đồ use case phân rã Quản lý phương tiện



Hình 2.6: Biểu đồ use case phân rã Quản lý phương tiện

Tìm kiếm phương tiện: Hỗ trợ quản lý tòa nhà và Ban quản trị lọc và tra cứu nhanh thông tin của các xe đăng ký trong tòa nhà.

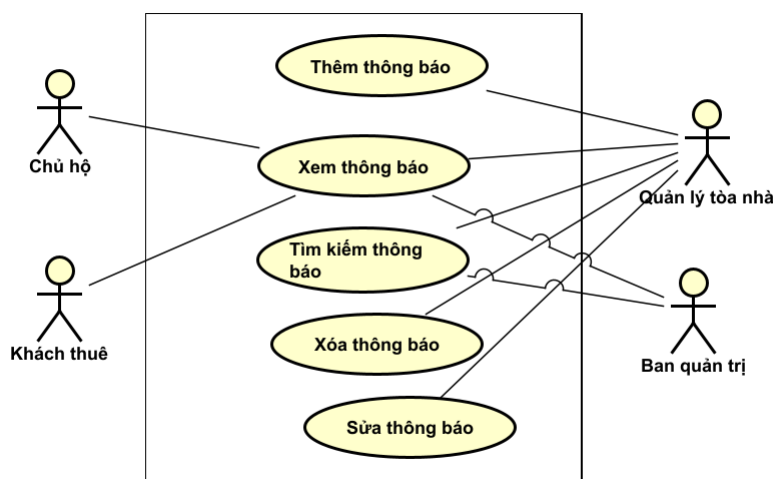
Xem thông tin phương tiện: Cho phép ban quản trị, quản lý tòa nhà tra cứu, theo dõi thông tin chi tiết toàn bộ phương tiện, cư dân chỉ có thể xem thông tin của căn hộ mình.

Thêm phương tiện: Cư dân khai báo phương tiện mới thuộc về căn hộ mình.

Sửa thông tin phương tiện: Cập nhật lại thông tin phương tiện đã đăng ký.

Xóa phương tiện: Xóa phương tiện đã đăng ký trong hệ thống.

2.2.7 Biểu đồ use case phân rã Quản lý thông báo



Hình 2.7: Biểu đồ use case phân rã Quản lý thông báo

Thêm thông báo: Cho phép quản lý tòa nhà tạo và gửi các thông báo mới gửi tới các căn hộ.

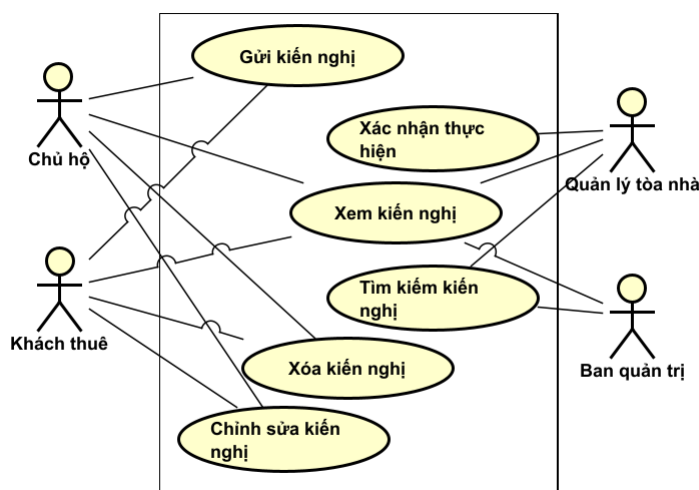
Xem thông báo: Cho phép ban quản trị, quản lý tòa nhà tra cứu, theo dõi toàn bộ thông báo, cư dân chỉ có thể xem thông báo của căn hộ mình.

Tìm kiếm thông báo: Tra cứu các thông báo đã gửi đi trong hệ thống.

Xóa thông báo: Xóa các thông báo khỏi hệ thống.

Sửa thông báo: Cập nhật lại nội dung của các thông báo đã phát hành.

2.2.8 Biểu đồ use case phân rã Quản lý kiến nghị



Hình 2.8: Biểu đồ use case phân rã Quản lý kiến nghị

Gửi kiến nghị: Cư dân gửi các ý kiến đóng góp, phản ánh hoặc khiếu nại lên ban quản lý.

Xác nhận thực hiện: Cho phép Quản lý tòa nhà cập nhật tiến độ giải quyết các

kiến nghị của cư dân.

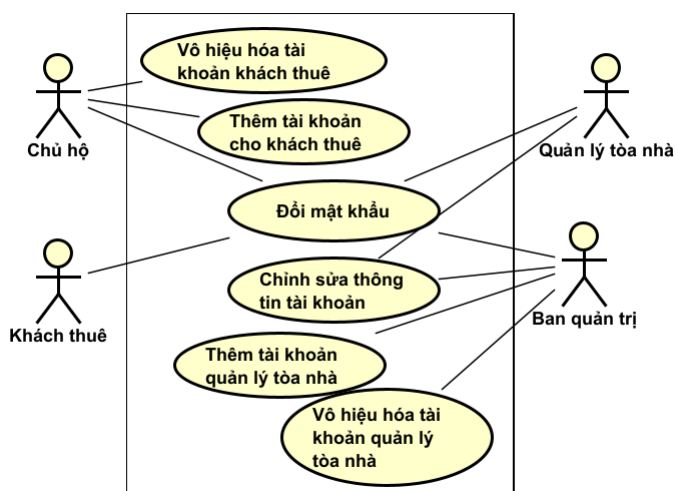
Xem kiến nghị: Theo dõi chi tiết nội dung và trạng thái xử lý của các kiến nghị.

Tìm kiếm kiến nghị: Hỗ trợ ban quản trị và quản lý tòa nhà lọc, tra cứu các kiến nghị trong hệ thống.

Xóa kiến nghị: Xóa các kiến nghị đã được gửi đi trong hệ thống.

Chỉnh sửa kiến nghị: Cập nhật lại nội dung phản ánh khi đơn kiến nghị chưa được ban quản lý phê duyệt hoặc xử lý.

2.2.9 Biểu đồ use case phân rã Quản lý tài khoản



Hình 2.9: Biểu đồ use case phân rã Quản lý tài khoản

Vô hiệu hóa tài khoản khách thuê: Cho phép chủ hộ khóa quyền truy cập hệ thống của tài khoản khách thuê.

Thêm tài khoản cho khách thuê: Hỗ trợ Chủ hộ tạo và cấp tài khoản đăng nhập mới cho người thuê căn hộ.

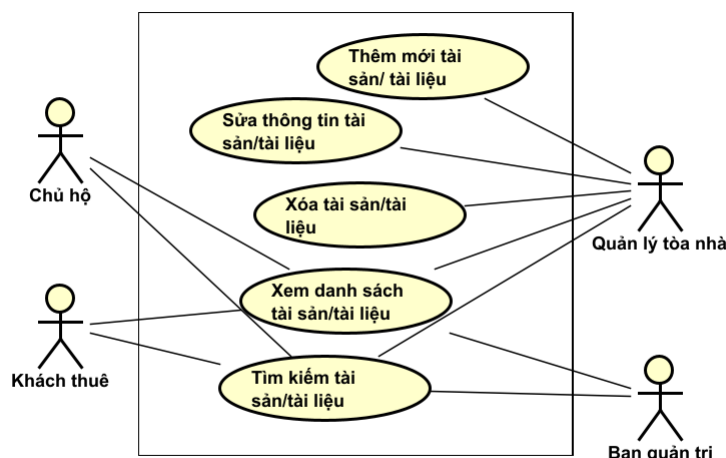
Đổi mật khẩu: Thay đổi mật khẩu dùng để đăng nhập vào hệ thống.

Chỉnh sửa thông tin tài khoản: Cho phép quản lý tòa nhà và ban quản trị cập nhật thông tin tài khoản của mình.

Thêm tài khoản quản lý tòa nhà: Cho phép Ban quản trị khởi tạo và cấp quyền truy cập hệ thống cho nhân sự thuộc ban quản lý tòa nhà mới.

Vô hiệu hóa tài khoản quản lý tòa nhà: Hỗ trợ Ban quản trị khóa quyền truy cập của tài khoản quản lý tòa nhà.

2.2.10 Biểu đồ use case phân rã Quản lý tài sản và tài liệu



Hình 2.10: Biểu đồ use case phân rã Quản lý tài sản và tài liệu

Thêm mới tài sản/tài liệu: Cho phép Quản lý tòa nhà đăng ký, lưu trữ thông tin về tài sản chung cư hoặc tài liệu của tòa nhà vào hệ thống.

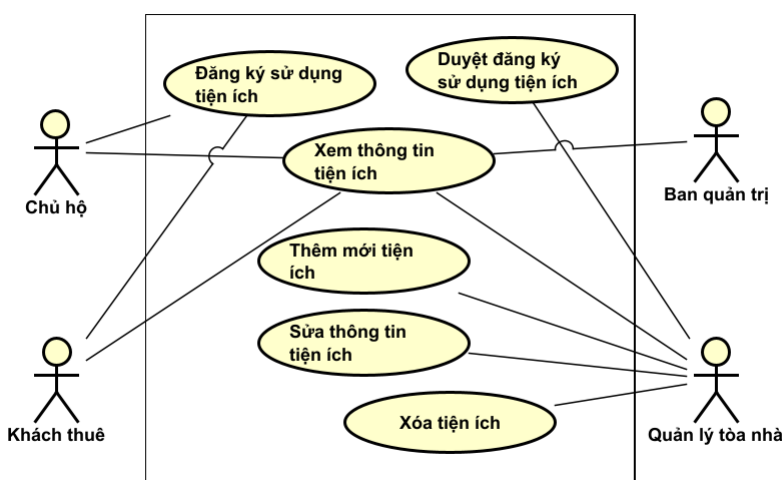
Sửa thông tin tài sản/tài liệu: Cập nhật, thay đổi các thông tin của tài sản và tài liệu đã lưu trữ.

Xóa tài sản/tài liệu: Xóa bỏ thông tin tài sản hoặc tài liệu ra khỏi danh mục quản lý.

Xem danh sách tài sản/tài liệu: Hiển thị toàn bộ danh mục tài sản hoặc tài liệu dùng chung được công khai.

Tìm kiếm tài sản/tài liệu: Cho phép người dùng tìm kiếm các tài liệu hoặc tài sản theo tên.

2.2.11 Biểu đồ use case phân rã Quản lý tiện ích



Hình 2.11: Biểu đồ use case phân rã Quản lý tiện ích

Đăng ký sử dụng tiện ích: Cư dân đặt lịch đăng ký sử dụng các tiện ích chung cư.

Duyệt đăng ký sử dụng tiện ích: Cho phép quản lý tòa nhà tiếp nhận, phê duyệt hoặc từ chối các yêu cầu đặt lịch sử dụng tiện ích của cư dân.

Xem thông tin tiện ích: Xem thông tin chi tiết của các tiện ích đang hoạt động của chung cư.

Thêm mới tiện ích: Cho phép quản lý tòa nhà thêm các tiện ích mới vào hệ thống quản lý.

Sửa thông tin tiện ích: Hỗ trợ quản lý tòa nhà điều chỉnh thông tin, số lượng tối đa hoặc giờ đóng mở cửa của các tiện ích hiện có.

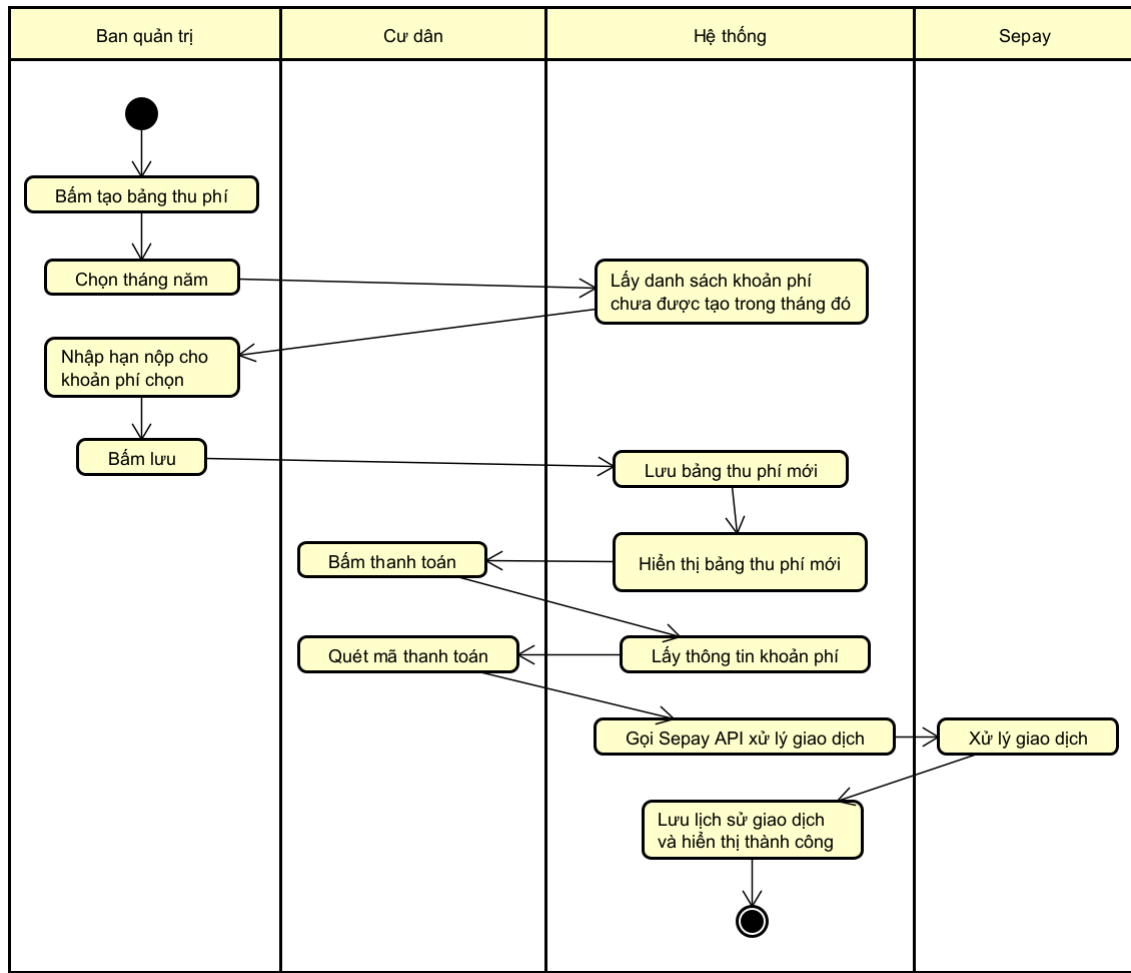
Xóa tiện ích: Cho phép quản lý tòa nhà xóa bỏ các tiện ích không còn được sử dụng tại tòa nhà.

2.2.12 Quy trình nghiệp vụ

Quy trình nghiệp vụ tổng thể về quản lý tài chính và thu chi trong hệ thống bao gồm chuỗi các hoạt động liên kết chặt chẽ và xuyên suốt giữa ba tác nhân là Ban quản trị, Cư dân và Hệ thống trung gian thanh toán SePay. Quy trình này kết hợp các chức năng từ khởi tạo dữ liệu cho đến đối soát và xác nhận dòng tiền tự động.

Đầu tiên là giai đoạn khởi tạo và phát hành bảng phí, định kỳ hàng tháng Ban quản trị truy cập vào hệ thống để thiết lập các khoản thu. Hệ thống sẽ tự động kiểm tra tính hợp lệ và truy xuất các danh mục chưa được tạo trong tháng đó. Sau khi Ban quản trị nhập hạn nộp và phê duyệt lệnh lưu, hệ thống sẽ chốt số liệu và tự động tạo các bản ghi hóa đơn dưới trạng thái chưa thanh toán. Tiếp theo là giai đoạn tra cứu và tương tác của cư dân, hệ thống tự động cập nhật bảng thu phí công khai lên ứng dụng di động của từng căn hộ, lúc này cư dân đăng nhập vào ứng dụng để kiểm tra chi tiết các mục tiền cần đóng và lựa chọn hóa đơn để thực hiện lệnh thanh toán. Cuối cùng là giai đoạn thanh toán và đối soát tự động, khi cư dân chọn thanh toán thì hệ thống sẽ kết nối với SePay API để sinh mã QR Code động chứa chính xác số tiền cùng mã định danh hóa đơn ngầm. Cư dân thực hiện quét mã bằng ứng dụng ngân hàng để chuyển tiền. Ngay sau khi giao dịch thành công, SePay gửi tín hiệu Webhook phản hồi ngầm về máy chủ Backend, hệ thống lập tức đối soát nội dung để chuyển trạng thái hóa đơn thành đã thanh toán, lưu vết lịch sử giao dịch và trả thông báo kết quả thời gian thực về giao diện của cư dân.

Dưới đây là sơ đồ mô tả chi tiết toàn bộ luồng hoạt động tích hợp của quy trình nghiệp vụ này:



Hình 2.12: Biểu đồ hoạt động nghiệp vụ tạo khoản phí và thanh toán phí

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case Lấy lại mật khẩu

Tên Use case	Lấy lại mật khẩu
Tác nhân	Ban Quản Trị, Quản lý tòa nhà, Chủ hộ, Khách thuê
Mô tả	Người dùng chọn lấy lại mật khẩu khi quên mật khẩu đăng nhập bằng cách xác thực mã OTP qua Email
Tiền điều kiện	Người dùng đã có email đăng ký tài khoản trên hệ thống
Luồng sự kiện chính	1. Người dùng chọn “Quên mật khẩu” trên màn hình đăng nhập 2. Hệ thống yêu cầu nhập email đã đăng ký 3. Người dùng nhập email và bấm "Gửi yêu cầu" 4. Hệ thống kiểm tra email tồn tại trong cơ sở dữ liệu 5. Hệ thống tạo mã OTP, lưu vào bộ nhớ tạm và gửi mã OTP về email của người dùng 6. Hệ thống hiển thị màn hình nhập mã OTP 7. Người dùng kiểm tra email, nhập mã OTP vào hệ thống và bấm "Xác nhận" 8. Hệ thống xác thực mã OTP hợp lệ và hiển thị màn hình nhập mật khẩu mới 9. Người dùng nhập mật khẩu mới, xác nhận lại mật khẩu và bấm "Lưu" 10. Hệ thống cập nhật mật khẩu mới vào cơ sở dữ liệu và hiển thị thông báo: “Đổi mật khẩu thành công”
Luồng sự kiện thay thế	4a. Nếu email không tồn tại trong hệ thống: Hệ thống hiển thị thông báo: “Email không tồn tại trên hệ thống” 7a. Nếu mã OTP người dùng nhập không chính xác: Hệ thống hiển thị thông báo: “Mã OTP không hợp lệ, vui lòng thử lại” 7b. Nếu mã OTP đã quá thời gian hiệu lực: Hệ thống hiển thị thông báo: “Mã OTP đã hết hạn” và cho phép người dùng bấm nút "Gửi lại mã"
Hậu điều kiện	Mật khẩu mới của người dùng được cập nhật thành công và người dùng có thể đăng nhập bằng mật khẩu mới

2.3.2 Đặc tả use case Thanh toán khoản phí

Tên Use case	Thanh toán
Tác nhân	Chủ hộ, khách thuê
Mô tả	Thanh toán các khoản phí
Tiền điều kiện	Cư dân đã đăng nhập vào hệ thống và có ít nhất một khoản phí chưa thanh toán
Luồng sự kiện chính	1. Cư dân vào mục Quản lý khoản phí 2. Hệ thống hiển thị danh sách khoản phí của căn hộ 3. Cư dân chọn khoản phí chưa thanh toán và ấn nút thanh toán 4. Hệ thống hiển thị thông tin thanh toán 5. Cư dân tiến hành quét mã để thanh toán 6. Hệ thống xử lý giao dịch và cập nhật trạng thái thanh toán 7. Hệ thống hiển thị thông báo thanh toán thành công
Luồng sự kiện thay thế	5a. Nếu cư dân hủy thanh toán: Hệ thống quay lại bước 2, không cập nhật trạng thái khoản phí 6a. Nếu giao dịch thất bại: Hệ thống hiển thị thông báo lỗi và trở lại bước 4
Hậu điều kiện	Khoản phí được cập nhật trạng thái đã thanh toán

2.3.3 Đặc tả use case Tạo bảng thu phí

Mã Use case	UC03
Tên Use case	Tạo bảng thu phí
Tác nhân	Quản lý tòa nhà
Mô tả	Tạo bảng tổng hợp các khoản thu phí từng tháng
Tiền điều kiện	Quản lý tòa nhà đã đăng nhập hệ thống
Luồng sự kiện chính	1. Quản lý tòa nhà truy cập vào mục Quản lý khoản phí 2. Quản lý tòa nhà chọn chức năng “Tạo bảng thu phí” 3. Hệ thống hiển thị form nhập tháng, năm 4. Quản lý tòa nhà nhập tháng, năm 5. Hệ thống hiển thị danh sách các loại phí chưa chọn 6. Ban quản trị chọn loại phí 7. Hệ thống hiển thị danh sách mã căn hộ, tên khoản phí đã chọn và hạn nộp cần nhập 8. Ban quản trị nhập hạn nộp 9. Hệ thống lặp lại bước 5-9 cho mỗi loại phí mới 10. Sau khi hoàn tất, Ban quản trị nhấn nút “Lưu” 11. Hệ thống lưu dữ liệu và hiển thị danh sách tổng hợp các khoản phí đã nhập cho tháng mới nhất
Luồng sự kiện thay thế	5a. Nếu Ban quản trị tạo bảng thu phí cho tất cả các loại phí của tháng đó: Hệ thống hiển thị danh sách trống 11a. Nếu lưu dữ liệu thất bại: Hệ thống hiển thị thông báo lỗi: "Lưu bảng phí thất bại. Vui lòng thử lại."
Hậu điều kiện	Bảng thu phí tháng mới được tạo và lưu trữ trong hệ thống và hiển thị trên giao diện

2.3.4 Đặc tả use case Thêm mới tài khoản căn hộ

Tên Use case	Thêm tài khoản vào căn hộ
Tác nhân	Quản lý tòa nhà
Mô tả	Quản lý tòa nhà tạo tài khoản đăng nhập cho một căn hộ
Tiền điều kiện	Ban quản trị đã đăng nhập thành công vào hệ thống
Luồng sự kiện chính	1. Quản lý tòa nhà truy cập vào trang quản lý căn hộ 2. Hệ thống hiển thị danh sách các căn hộ của tòa nhà 3. Quản lý tòa nhà ấn nút thêm tài khoản 4. Hệ thống hiển thị form chọn loại tài khoản mới hoặc đã có 5. Quản lý tòa nhà chọn tùy chọn "Tạo tài khoản mới" 6. Quản lý tòa nhà nhập đủ thông tin của form tạo mới tài khoản 7. Quản lý tòa nhà nhấn "Lưu" 8. Hệ thống kiểm tra dữ liệu, tạo tài khoản mới, cập nhật trạng thái căn hộ thành đã bán, tạo một bản ghi mới vào bảng "Lịch sử cư trú" của căn hộ đó 9. Hệ thống hiển thị thông báo: "Thêm tài khoản thành công"
Luồng sự kiện thay thế	3a. Nếu quản lý tòa nhà chọn thêm tài khoản cho căn hộ đã có tài khoản sẽ báo lỗi: "Căn hộ đã có tài khoản" 5a. Quản lý tòa nhà chọn tùy chọn "Chọn tài khoản có sẵn" 5a1. Quản lý tòa nhà nhập email và chủ hộ vào form. 5a2. Hệ thống kiểm tra và liên kết tài khoản này với căn hộ mới, tiếp tục đến bước 7 6-8a. Nếu thông tin nhập vào không đúng định dạng hoặc email đã bị trùng: Hệ thống hiển thị thông báo lỗi 7a. Nếu quản lý tòa nhà nhấn nút "Hủy": Hệ thống hủy bỏ mọi thay đổi và quay lại màn hình danh sách căn hộ
Hậu điều kiện	Căn hộ được chuyển sang trạng thái đã bán, bảng "Lịch sử cư trú" thêm bản ghi mới và cư dân có thể dùng tài khoản vừa tạo để đăng nhập và xem thông tin căn hộ

2.3.5 Đặc tả use case Vô hiệu hóa tài khoản căn hộ

Tên Use case	Vô hiệu hóa tài khoản căn hộ
Tác nhân	Quản lý tòa nhà
Mô tả	Vô hiệu hóa tài khoản căn hộ khi chủ hộ chuyển đi
Tiền điều kiện	Ban quản trị đã đăng nhập vào hệ thống
Luồng sự kiện chính	1. Quản lý tòa nhà truy cập chức năng Quản lý căn hộ 2. Hệ thống hiển thị danh sách căn hộ với trạng thái 3. Quản lý tòa nhà chọn tài khoản và nhấn nút “Khóa” 4. Hệ thống hiển thị hộp thoại xác nhận thao tác 5. Quản lý tòa nhà xác nhận 6. Hệ thống cập nhật trạng thái căn hộ thành “inactive” và xóa thông tin liên quan tới chủ hộ tại danh sách căn hộ 7. Hệ thống ghi nhận thời điểm khóa vào bản ghi của tài khoản chủ hộ trong bảng "Lịch sử cư trú" của căn hộ đó 8. Hệ thống ẩn cư dân, phương tiện, kiến nghị của căn hộ 9. Hệ thống hiển thị: “Khóa tài khoản thành công”
Luồng sự kiện thay thế	3a. Nếu quản lý tòa nhà ấn khóa tài khoản căn hộ chưa có tài khoản: Hiển thị lỗi "Căn hộ chưa có tài khoản" 5a. Nếu quản lý tòa nhà ấn hủy: Không thay đổi, quay lại danh sách căn hộ
Hậu điều kiện	Tài khoản bị khóa không thể đăng nhập, trạng thái căn hộ chuyển thành inactive và thông tin chủ hộ được lưu vào lịch sử cư trú của căn hộ, hệ thống không còn hiển thị danh sách cư dân, phương tiện, kiến nghị, đăng ký tiện ích của căn hộ đó

2.4 Yêu cầu phi chức năng

Hiệu suất

Hệ thống có thời gian phản hồi nhanh chóng, đáp ứng nhu cầu xử lý dữ liệu và truy cập thông tin của người dùng. Khả năng xử lý tải cao hiệu quả, đảm bảo hệ thống hoạt động ổn định trong các trường hợp lượng truy cập tăng đột biến.

Bảo mật

Bảo vệ dữ liệu khỏi truy cập trái phép, rò rỉ thông tin và các tấn công mạng. Xác thực và ủy quyền truy cập hệ thống cho người dùng dựa trên vai trò và quyền hạn.

Khả năng sử dụng

Giao diện hệ thống đơn giản, trực quan, dễ dàng sử dụng cho mọi đối tượng người dùng.

Tính di động

Hệ thống có thể hoạt động trên nhiều nền tảng khác nhau như máy tính, điện thoại thông minh, máy tính bảng. Giao diện hệ thống tự động thích ứng với kích thước màn hình của thiết bị.

Khả năng bảo trì

Hệ thống dễ dàng bảo trì, sửa lỗi và cập nhật phiên bản mới. Cấu trúc mã nguồn rõ ràng, logic, dễ dàng kiểm tra và sửa chữa.

Khả năng phục hồi

Hệ thống có khả năng phục hồi nhanh chóng trong trường hợp xảy ra lỗi hoặc sự cố. Sao lưu dữ liệu định kỳ và có kế hoạch phòng ngừa rủi ro.

Tổng kết lại, Chương 2 đã làm rõ nhu cầu người dùng và các tác nhân trực tiếp tham gia vào hệ thống thông qua việc khảo sát hiện trạng, xây dựng biểu đồ ca sử dụng tổng quát và biểu đồ ca sử dụng phân rã và đặc tả chi tiết các ca sử dụng chính, phân tích các ràng buộc phi chức năng về hiệu năng và bảo mật. Các khảo sát trong chương này làm tiền đề vững chắc cho việc triển khai kiến trúc phần mềm ở chương tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này tập trung hệ thống hóa, phân tích và tóm tắt toàn bộ các nền tảng công nghệ, thư viện, kiến trúc phần mềm và giải pháp kỹ thuật được áp dụng để xây dựng hệ thống quản lý chung cư. Nội dung trình bày đảm bảo tính nhất quán, liên kết chặt chẽ với các yêu cầu hệ thống đã đặt ra. Với mỗi công nghệ được lựa chọn, báo cáo sẽ làm rõ ưu điểm, vai trò giải quyết bài toán cụ thể và đối sánh với các phương án thay thế tương đương nhằm chứng minh tính khoa học và tối ưu của giải pháp.

3.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế và tổ chức mã nguồn theo kiến trúc Client-Server [1], ứng dụng nguyên lý phân tầng chức năng tương tự mô hình MVC (Model - View - Controller). Việc áp dụng kiến trúc này giúp phân tách hệ thống thành các khối độc lập, có ranh giới rõ ràng, từ đó tối ưu hóa tốc độ thực thi, nâng cao khả năng bảo trì và tái sử dụng mã nguồn.

Lớp dữ liệu (Model): Đóng vai trò định nghĩa cấu trúc dữ liệu, các ràng buộc toàn vẹn và mối quan hệ cốt lõi của bài toán quản lý chung cư (như thông tin căn hộ, cư dân, các biểu phí dịch vụ lũy tiến). Thành phần này tương tác trực tiếp với hệ quản trị cơ sở dữ liệu thông qua tầng giao tiếp dữ liệu an toàn (ORM), đảm bảo mọi truy vấn đều được tham số hóa để phòng chống các lỗ hổng an ninh [2].

Lớp điều khiển (Controller/API Provider): Là thành phần trung tâm chịu trách nhiệm điều hướng luồng xử lý của toàn bộ hệ thống, được hiện thực hóa bằng Django REST Framework [3]. Tầng này tiếp nhận các yêu cầu HTTP Request gửi lên từ Client, thực thi các logic nghiệp vụ (như kiểm tra quyền hạn truy cập RBAC, xác thực mã JWT, xử lý logic thanh toán qua SePay Webhook), sau đó điều phối, tuần tự hóa và đóng gói dữ liệu dưới dạng JSON để trả về cho lớp hiển thị.

Lớp hiển thị (View/Client): Đảm nhận trách nhiệm trình bày giao diện trực quan và tương tác với người dùng cuối. Trong đồ án, lớp hiển thị được tách thành hai phân hệ độc lập là giao diện Web (React) [4] và ứng dụng di động (React Native) [5]. Hai phân hệ này tiếp nhận dữ liệu định dạng JSON được chuẩn hóa từ Backend để tự động hiển thị giao diện phù hợp với từng loại thiết bị.

Khi so sánh với kiến trúc Microservices — một xu hướng chia nhỏ hệ thống thành các dịch vụ độc lập kết nối qua API để tối ưu hóa việc mở rộng liên tục — mô hình này tuy hiện đại nhưng lại khá cồng kềnh, dễ làm phân tán dữ liệu và đòi hỏi cấu hình hạ tầng tốn kém. Đối với một hệ thống quản lý nội bộ tòa nhà, việc áp

dụng kiến trúc Monolith kết hợp tách rời Client-Server như trên là giải pháp tối ưu hiện tại. Kiến trúc này giúp giữ mã nguồn backend tập trung, tận dụng tối đa khả năng xử lý dữ liệu và các rào chắn bảo mật tích hợp sẵn của Django ở phía máy chủ [2], đồng thời vẫn đảm bảo tính linh hoạt, mượt mà khi cấp phát dữ liệu song song cho đa nền tảng Client.

3.2 Nền tảng Backend và hệ quản trị cơ sở dữ liệu

3.2.1 Nền tảng Backend

Hệ thống sử dụng cấu trúc Django kết hợp với Django REST Framework làm nền tảng cốt lõi để xây dựng các thành phần dịch vụ phía máy chủ [3]. Trong kiến trúc này, hệ thống ánh xạ thực thể mối quan hệ đóng vai trò quản lý tầng dữ liệu, giúp thao tác với cơ sở dữ liệu một cách an toàn và ngăn chặn các lỗ hổng bảo mật phổ biến [2]. Tầng điều khiển được hiện thực hóa thông qua các View và Serializer của DRF, đảm nhận nhiệm vụ tiếp nhận yêu cầu, tuần tự hóa dữ liệu và điều phối luồng xử lý logic trước khi phản hồi về phía giao diện [3].

Khi xem xét các giải pháp thay thế như Node.js (Express) hay Java (Spring Boot), Django và DRF bộc lộ những điểm phù hợp hơn cho dự án. Trong khi Spring Boot đòi hỏi cấu hình phức tạp với thời gian phát triển kéo dài, còn Node.js tuy mạnh về xử lý I/O nhưng lại thiếu cấu trúc phân tầng chuẩn hóa, thì Django cung cấp một kiến trúc phù hợp hơn cả. Kiến trúc này giúp tổ chức và triển khai các thành phần Model và Controller một cách nhanh chóng, đồng bộ và nhất quán [3].

Để đảm bảo an toàn cho các luồng dữ liệu luân chuyển qua tầng Controller, hệ thống tích hợp thư viện Djoser cùng SimpleJWT để chuẩn hóa toàn bộ quy trình xác thực người dùng. Thay vì sử dụng cơ chế định danh truyền thống, hệ thống áp dụng phương thức xác thực mã thông báo định hạn (JWT) với access token và refresh token có thiết lập tự động xoay vòng và đưa vào danh sách đen sau khi sử dụng để ngăn chặn các nguy cơ chiếm đoạt phiên. Bên cạnh đó, hệ thống kết hợp cấu hình thiết lập Cookie. Cơ chế này giúp bảo vệ mã thông báo trực tiếp trên trình duyệt hoặc ứng dụng phía máy trạm, giảm thiểu rủi ro từ các cuộc tấn công giả mạo yêu cầu chéo trường (CSRF) và kịch bản chèn mã độc mã nguồn giao diện (XSS) [2].

Đối với việc quản lý các biến môi trường và thông tin cấu hình, thư viện Python-dotenv được lựa chọn để đảm bảo tính an toàn cho hệ thống [6]. Thư viện này đảm nhận vai trò tách biệt toàn bộ các thông tin cấu hình nhạy cảm, thông tin kết nối cơ sở dữ liệu của tầng Model, cùng các khóa bí mật kết nối dịch vụ bên ngoài ra khỏi mã nguồn gốc. Toàn bộ dữ liệu cấu hình được lưu trữ riêng biệt trong tệp .env, giúp

kiểm soát quyền truy cập chặt chẽ và ngăn ngừa nguy cơ rò rỉ khi mã nguồn được lưu trữ trên GitHub.

3.2.2 Hệ quản trị cơ sở dữ liệu

Hệ thống lựa chọn PostgreSQL làm hệ quản trị cơ sở dữ liệu quan hệ lõi, đóng vai trò nền tảng cho việc lưu trữ và quản lý toàn bộ thực thể thông tin trong tầng Model [7]. Giải pháp này đảm bảo tính toàn vẹn dữ liệu thông qua cơ chế ràng buộc khóa ngoại chặt chẽ, tuân thủ các nguyên tắc ACID trong giao dịch, đồng thời cung cấp khả năng xử lý các câu lệnh truy vấn cấu trúc phức tạp và hỗ trợ định dạng dữ liệu JSONB linh hoạt.

So với các giải pháp thay thế như MySQL hay cơ sở dữ liệu phi quan hệ MongoDB, PostgreSQL bộc lộ những ưu thế phù hợp rõ rệt với đặc thù nghiệp vụ của dự án [7]. Cụ thể, cấu trúc không định hình của MongoDB không phù hợp cho các bài toán quản lý tài chính và hóa đơn dịch vụ chung cư vốn đòi hỏi tính liên kết và ràng buộc quan hệ chặt chẽ giữa các bảng dữ liệu. Đối với MySQL, khi đối mặt với lượng dữ liệu lịch sử tiêu thụ điện, nước tích lũy theo thời gian của tòa nhà, PostgreSQL thể hiện sự vượt trội về khả năng xử lý tác vụ ghi đồng thời cũng như hiệu năng tối ưu hóa chỉ mục nâng cao, giúp đảm bảo tốc độ phản hồi ổn định cho hệ thống.

3.3 Nền tảng Frontend

3.3.1 Web

Web đóng vai trò là tầng giao diện trực quan, đảm nhận nhiệm vụ hiển thị thông tin, quản lý các bảng dữ liệu quy mô lớn và tương tác liên tục với tầng điều khiển của Backend. Để đáp ứng nhu cầu sử dụng của cả hai nhóm đối tượng là ban quản trị và cư dân, giao diện được xây dựng trên cấu trúc phân quyền chặt chẽ. Hệ thống sử dụng thư viện React kết hợp với React Router để tổ chức các thành phần độc lập, đồng thời thiết lập các tuyến đường bảo vệ [4]. Cơ chế Virtual DOM của React giúp tối ưu hóa tốc độ cập nhật giao diện, trong khi React Router hỗ trợ chuyển đổi linh hoạt giao diện một cách tức thì mà không cần tải lại toàn bộ trang web [4].

Về giải pháp thay thế, thay vì sử dụng kiến trúc đa trang truyền thống (MPA) với các framework như Django Templates, hệ thống quyết định lựa chọn mô hình Single Page Application (SPA) của React [4]. Lý do là vì kiến trúc MPA bắt buộc trình duyệt phải tải lại toàn bộ trang và thực thi lại các truy vấn giao diện mỗi khi người dùng chuyển mục, gây ra độ trễ lớn và làm giảm trải nghiệm người dùng đối với một hệ thống quản lý tòa nhà cần tương tác dữ liệu liên tục. React SPA giúp khắc phục hoàn toàn nhược điểm này bằng cách chỉ tải tài nguyên một lần duy nhất và cập nhật động các thành phần giao diện khi có thay đổi dữ liệu [4].

Quản lý trạng thái tập trung toàn cầu được đảm nhiệm bởi Redux Toolkit. Thư viện này giải quyết hiệu quả bài toán đồng bộ dữ liệu giữa các thành phần nằm xa nhau trên cây giao diện, đặc biệt là thông tin định danh và vai trò của tài khoản đang đăng nhập. Dựa vào trạng thái vai trò được lưu trữ tập trung này, giao diện sẽ tự động quyết định quyền hiển thị thanh điều hướng, các chức năng quản lý riêng cho ban quản trị, hoặc giao diện đóng tiền, gửi kiến nghị dành riêng cho cư dân, từ đó loại bỏ hiện tượng truyền dữ liệu phức tạp qua nhiều cấp bậc trung gian.

Giải pháp React Toolkit được lựa chọn thay vì sử dụng Context API mặc định của React, do Context API thường gây ra hiện tượng tái kết xuất (re-render) toàn bộ các thành phần con một cách không cần thiết khi có một trạng thái thay đổi, làm suy giảm hiệu năng hiển thị của các bảng thống kê dữ liệu lớn.

Để kết nối và khai thác các dịch vụ từ Controller, hệ thống sử dụng thư viện HTTP Client Axios [8]. Thư viện này được cấu hình thuộc tính withCredentials nhằm tự động đính kèm cặp mã xác thực JWT lưu trữ trong Cookie an toàn của trình duyệt vào mỗi yêu cầu gửi đi. Đồng thời, hệ thống tận dụng cơ chế Interceptor của Axios để xử lý tập trung các lỗi phản hồi từ server, kiểm tra quyền truy cập hợp lệ của từng tài khoản và tự động gửi yêu cầu làm mới mã thông báo khi access token hết hạn, đảm bảo phiên làm việc của cả ban quản trị lẫn cư dân không bị gián đoạn [8].

Ngôn ngữ SCSS được lựa chọn làm bộ tiền xử lý CSS, giúp tổ chức mã nguồn định dạng theo cấu trúc phân cấp, dễ dàng tùy biến giao diện Responsive tương thích tốt trên cả màn hình máy tính của ban quản trị lẫn màn hình trình duyệt điện thoại của cư dân [9]. Thư viện Chart.js được tích hợp để chuyển đổi các số liệu thô thành biểu đồ nhằm phục vụ thống kê tài chính.

3.3.2 Ứng dụng di động

Ứng dụng di động đóng vai trò là tầng giao diện mở rộng, có nhiệm vụ tiếp nhận và hiển thị dữ liệu được chuẩn hóa từ tầng Serializer của Backend. Hệ thống áp dụng React Native kết hợp với hệ sinh thái Expo để phát triển ứng dụng đa nền tảng từ một kho mã nguồn duy nhất [5]. Lựa chọn này giúp rút ngắn thời gian phát triển so với phương pháp Native thuần, tận dụng các thư viện liên kết phần cứng chuẩn hóa của Expo mà không cần can thiệp sâu vào cấu hình hệ thống phức tạp [5].

Khi cân nhắc các giải pháp thay thế, hệ thống đã loại bỏ phương án phát triển ứng dụng Native thuần (Sử dụng Swift cho iOS và Kotlin cho Android). Mặc dù ứng dụng Native thuần mang lại hiệu năng tối đa nhưng lại yêu cầu chi phí vận hành và thời gian phát triển gấp đôi để duy trì hai kho mã nguồn độc lập. React Native

kết hợp Expo là sự cân bằng tối ưu nhờ khả năng biên dịch mã nguồn JavaScript thành các thành phần giao diện Native thực tế của hệ điều hành, đảm bảo hiệu năng hoạt động mượt mà trong khi vẫn tối ưu hóa được thời gian phát triển trên một mã nguồn duy nhất [5].

Để nâng cao độ tin cậy của mã nguồn, ngôn ngữ TypeScript được đưa vào sử dụng nhằm bổ sung cơ chế kiểm tra kiểu dữ liệu tĩnh. Giải pháp này giúp phát hiện sớm các lỗi cú pháp và sự sai lệch cấu trúc dữ liệu ngay trong quá trình phát triển, đảm bảo tính chính xác cho các thông tin nhạy cảm như hóa đơn hay chi phí dịch vụ trên ứng dụng.

Luồng trải nghiệm người dùng trên thiết bị di động được điều phối thông qua hai công cụ chuyên dụng là Async Storage và React Navigation [5]. Trong đó, Async Storage đóng vai trò là hệ thống lưu trữ cục bộ dạng khóa - giá trị để lưu giữ mã xác thực, giúp duy trì trạng thái đăng nhập cho các phiên làm việc kế tiếp. React Navigation đảm nhận nhiệm vụ quản lý luồng chuyển màn hình, hỗ trợ xây dựng các mô hình điều hướng, mang lại trải nghiệm vuốt chạm mượt mà [5].

Redux Toolkit tiếp tục được áp dụng đồng bộ trên nền tảng di động để quản lý các trạng thái bất đồng bộ trả về từ API backend, giúp thống nhất về mặt lập trình và kiến trúc phát triển với Web.

3.4 Cơ chế bộ nhớ đệm và tối ưu hóa hiệu năng

Để đáp ứng yêu cầu phi chức năng về hiệu năng cao, tối ưu hóa tốc độ phản hồi tại tầng điều khiển và ngăn chặn tình trạng nghẽn máy chủ khi xử lý lượng truy cập lớn, hệ thống tích hợp hạ tầng lưu trữ đệm trung gian và xử lý tác vụ bất đồng bộ dựa trên nền tảng Redis [10].

Về vấn đề tối ưu hóa truy vấn, hệ thống cấu hình Redis làm tầng lưu trữ đệm thông qua thư viện django-redis tại tầng Controller [3], [10]. Cơ chế này được áp dụng để lưu trữ kết quả của các yêu cầu API có tần suất truy cập cao từ cư dân nhưng ít khi biến đổi dữ liệu, chẳng hạn như thông tin bảng phí, thông báo chung hay danh sách tiện ích. Khi có các yêu cầu trùng lặp gửi đến cùng một API trong một khoảng thời gian nhất định, hệ thống sẽ lập tức trả về dữ liệu từ bộ nhớ đệm thay vì phải thực thi lại chuỗi câu lệnh SQL vào cơ sở dữ liệu PostgreSQL. Giải pháp này giúp giảm thiểu độ trễ phản hồi, tối ưu hóa băng thông luân chuyển và giảm tải đáng kể cho hệ quản trị cơ sở dữ liệu chính [7].

Bên cạnh đó, hệ thống tối ưu hóa hiệu năng tổng thể bằng cách chuyển giao các tác vụ tốn kém tài nguyên ra khỏi luồng xử lý phản hồi API chính nhờ vào việc tích hợp kiến trúc Celery Worker và Celery Beat. Trong môi trường phát triển hệ

thông, Redis được tận dụng song song làm trình trung chuyển tin nhắn (Message Broker) trung gian, kết nối linh hoạt giữa tầng ứng dụng Django và các tiến trình chạy ngầm. Đối với các tác vụ đòi hỏi thời gian xử lý kéo dài và dễ gây nghẽn kết nối như gửi email thông báo phí hàng loạt, tầng Controller sẽ lập tức đẩy thông điệp tác vụ vào hàng đợi Redis và trả về phản hồi thành công ngay lập tức cho người dùng. Phân hệ Celery Worker vận hành độc lập sẽ chủ động tiếp nhận gói tin từ hàng đợi này và thực thi xử lý ngầm ở tầng nền, giải phóng hoàn toàn thời gian chờ cho luồng Request-Response chính của máy chủ.

Song song với cơ chế xử lý bất đồng bộ theo yêu cầu, hệ thống tích hợp bộ lập lịch tự động Celery Beat để quản lý các tác vụ có tính chu kỳ và quản trị dữ liệu tòa nhà. Phân hệ này đóng vai trò như một bộ định thời hệ thống, liên tục kiểm tra và tự động kích hoạt các tiến trình ngầm theo các mốc thời gian được thiết lập sẵn, chẳng hạn như quét cơ sở dữ liệu vào khung giờ thấp điểm để tìm kiếm các khoản phí sắp đến hạn, tự động hóa chu trình tính toán công nợ lũy kế, hoặc khởi tạo lệnh nhắc nhở nộp phí định kỳ. Việc kết hợp chặt chẽ giữa Celery Beat để hoạch định thời gian và Celery Worker để thực thi tác vụ giúp hệ thống tự động hóa hoàn toàn các nghiệp vụ lặp đi lặp lại của Ban quản lý mà không gây ảnh hưởng đến trải nghiệm tương tác trực quan của cư dân trên giao diện di động.

Bên cạnh đó, việc áp dụng giải pháp lưu trữ trên bộ nhớ RAM của Redis giúp hệ thống tinh gọn hạ tầng kỹ thuật, mang lại tốc độ đọc ghi vượt trội với độ trễ tối thiểu [10]. Toàn bộ quá trình lưu trữ đệm, cấu hình thời gian hết hạn và cơ chế xóa bỏ dữ liệu đệm lỗi thời khi có cập nhật mới đều được thiết lập tại tầng mã nguồn backend [3]. Điều này đảm bảo tính nhất quán dữ liệu giữa bộ nhớ đệm và cơ sở dữ liệu gốc, giúp ban quản trị và cư dân luôn tiếp cận được thông tin chính xác, kịp thời trong suốt quá trình vận hành hệ thống tòa nhà.

Hệ thống lựa chọn Redis thay vì Memcached làm giải pháp lưu trữ đệm nhờ vào khả năng xử lý linh hoạt và độ an toàn dữ liệu vượt trội [10]. Trong khi Memcached chỉ hỗ trợ kiểu dữ liệu Key-Value buộc phải ghi đè toàn bộ đối tượng khi có thay đổi, Redis cung cấp nhiều cấu trúc dữ liệu phức tạp như Hashes, Lists hay Sets, cho phép hệ thống truy xuất và cập nhật trực tiếp từng trường thông tin nhỏ của cư dân hay căn hộ một cách nhanh chóng [10]. Đặc biệt, Redis có khả năng sao lưu dữ liệu xuống đĩa cứng thay vì chỉ lưu trên RAM như Memcached; điều này giúp hệ thống dễ dàng phục hồi dữ liệu đệm sau sự cố khởi động lại, ngăn chặn tình trạng hàng loạt truy vấn tràn xuống làm quá tải cơ sở dữ liệu PostgreSQL chính [7]. Ngoài ra, việc kết hợp Redis với hệ sinh thái Django thông qua thư viện django-redis mang lại khả năng tối ưu hóa kết nối và quản lý vòng đời bộ nhớ đệm chặt chẽ hơn [3],

[10], giúp hạ tầng kỹ thuật vận hành ổn định và đồng bộ hơn.

3.5 Giải pháp an ninh hệ thống và bảo mật dữ liệu

Hệ thống quản lý chung cư nắm giữ nhiều thông tin nhạy cảm về nhân khẩu và các giao dịch tài chính, do đó các giải pháp bảo mật được thiết kế đa tầng nghiêm ngặt, áp dụng đồng bộ từ tầng dữ liệu, tầng điều khiển cho đến tầng giao diện.

Để bảo vệ các luồng dữ liệu luân chuyển, hệ thống áp dụng cơ chế xác thực không lưu trạng thái thông qua chuẩn JSON Web Token với thư viện SimpleJWT tại tầng Controller [2]. Sau khi tài khoản đăng nhập thành công, máy chủ sẽ cấp phát một cặp mã access Token và refresh Token cho phía máy trạm. Cơ chế này loại bỏ rủi ro từ các cuộc tấn công chiếm đoạt phiên làm việc vật lý trên máy chủ. Đi kèm với mã xác thực là mô hình phân quyền kiểm soát truy cập dựa trên vai trò được cấu hình chặt chẽ. Hệ thống thiết lập ma trận quyền hạn rõ ràng cho từng nhóm đối tượng bao gồm Ban quản trị, quản lý tòa nhà, chủ hộ và khách thuê. Middleware của Django REST Framework sẽ chịu trách nhiệm kiểm tra quyền hạn gắn liền với mã thông báo của người dùng trước khi cho phép thực thi bất kỳ hành động nào, đảm bảo tính biệt lập dữ liệu [3].

Trong trường hợp người dùng cần khôi phục mật khẩu, hệ thống triển khai cơ chế xác thực hai lớp qua mã OTP kết hợp giao thức OAuth2 SMTP **celery_docs**. Khi có yêu cầu đặt lại mật khẩu từ giao diện, hệ thống tự động khởi tạo một mã OTP có giới hạn thời gian hiệu lực ngắn và lưu tạm thời vào bộ nhớ đệm [10]. Để chuyển giao mã này đến hòm thư điện tử của người dùng, hệ thống kết nối với dịch vụ Google Mail SMTP thông qua giao thức xác thực nâng cao OAuth2. Hướng tiếp cận này giúp loại bỏ việc lưu trữ cấu hình mật khẩu ứng dụng dạng văn bản thô trong mã nguồn, ngăn chặn nguy cơ bị đánh chặn luồng gửi thư hoặc rò rỉ tài khoản máy chủ. Người dùng bắt buộc phải cung cấp chính xác mã OTP nhận được để hệ thống xác thực danh tính tại Controller trước khi được cấp quyền thiết lập mật khẩu mới [3].

Đối với việc phòng chống các lỗ hổng bảo mật phổ biến, hệ thống xử lý triệt để nguy cơ tấn công chèn mã lệnh SQL (SQL Injection) bằng việc áp dụng Django ORM cho toàn bộ các thao tác truy vấn tại tầng Model [2]. Django ORM sử dụng cơ chế tham số hóa câu lệnh, tách biệt hoàn toàn phần câu lệnh thực thi của cơ sở dữ liệu và phần dữ liệu đầu vào từ người dùng. Mọi chuỗi ký tự do người dùng nhập vào đều được lớp Serializer ép kiểu dữ liệu cấu trúc, lọc bỏ các ký tự nguy hiểm và chỉ được xử lý như một giá trị tham số thuần túy, triệt tiêu khả năng thực thi mã lệnh ngoài ý muốn [3]. Bên cạnh đó, hệ thống cũng kích hoạt phân lớp kiểm soát mặc định của Django để phòng chống tấn công giả mạo yêu cầu chéo trường

(CSRF) đối với các yêu cầu thay đổi dữ liệu từ Web [2]. Đối với các luồng trao đổi API đa nền tảng kết nối với ứng dụng di động, hệ thống thiết lập cấu hình kiểm soát nguồn gốc (CORS) nghiêm ngặt tại tầng trung gian, chỉ chấp nhận các yêu cầu hợp lệ từ các tên miền và môi trường được định nghĩa sẵn trong cấu hình hệ thống.

3.6 Các giải pháp hỗ trợ

Hệ thống triển khai giải pháp tự động hóa thanh toán thông qua việc tích hợp SePay API, giải quyết triệt để bài toán gạch nợ hóa đơn tự động tại tầng dữ liệu [11]. Khi hệ thống khởi tạo hóa đơn phí dịch vụ mới, một mã QR thanh toán động theo chuẩn VietQR chứa nội dung chuyển khoản định danh duy nhất sẽ được sinh ra và hiển thị trên tầng giao diện. Ngay khi cư dân hoàn tất thao tác chuyển tiền qua ứng dụng ngân hàng, hệ thống SePay sẽ ghi nhận biến động số dư tài khoản và lập tức gửi tín hiệu bảo mật dạng Webhook về phía API của Backend [11]. Tầng Controller của Django khi tiếp nhận Webhook này sẽ tiến hành phân tích nội dung, đối khớp mã hóa đơn tại tầng nghiệp vụ và tự động cập nhật trạng thái bản ghi trong cơ sở dữ liệu PostgreSQL sang đã thanh toán và lưu lịch sử giao dịch thanh toán với thời gian phản hồi ngắn, loại bỏ hoàn toàn quy trình đối soát và xác minh thủ công từ ban quản trị [3], [7].

Bên cạnh giải pháp thanh toán, hệ thống sử dụng nền tảng điện toán đám mây Cloudinary để quản lý và lưu trữ toàn bộ dữ liệu đa phương tiện [12]. Nhằm ngăn ngừa nguy cơ cạn kiệt tài nguyên máy chủ do dung lượng lớn từ các tệp tin hình ảnh do cư dân đính kèm khi gửi kiến nghị, ảnh thẻ xe hay các văn bản quy định của tòa nhà, hệ thống tích hợp thư viện Cloudinary trực tiếp vào cấu hình lưu trữ của Django Backend [12]. Quy trình xử lý tệp tin được thiết lập để chuyển hướng toàn bộ dữ liệu tải lên thẳng đến kho lưu trữ của đám mây Cloudinary. Lúc này, tầng dữ liệu của cơ sở dữ liệu PostgreSQL chỉ thực hiện nhiệm vụ lưu trữ chuỗi đường dẫn URL liên kết tĩnh [7], giúp tối ưu hóa dung lượng ổ đĩa của máy chủ backend, đồng thời tăng tốc độ hiển thị tài nguyên trên giao diện nhờ vào mạng lưới phân phối nội dung toàn cầu của đối tác cung cấp [12].

Hệ thống tích hợp giải pháp trí tuệ nhân tạo Gemini Chatbot bằng thư viện google-genai tại tầng Controller. Tiện ích này cung cấp một kênh hỗ trợ trực tuyến tự động và phản hồi liên tục nhằm giải đáp thắc mắc của cư dân. Khi cư dân gửi câu hỏi, tầng Controller sẽ tiếp nhận thông tin, chuẩn hóa dữ liệu đầu vào và chuyển tiếp yêu cầu đến API của Google Gemini. Mô hình ngôn ngữ lớn sau khi tiếp nhận yêu cầu sẽ phân tích ngữ cảnh dựa trên bộ tài liệu quy chuẩn của tòa nhà được cấu hình sẵn để trả về câu trả lời chính xác và thân thiện. Giải pháp này giúp giảm bớt tần suất giải đáp các thắc mắc cho ban quản lý, đồng thời hỗ trợ cư dân tiếp cận

thông tin tiện ích một cách nhanh chóng và chủ động.

Tóm lại, toàn bộ cấu trúc công nghệ được lựa chọn trong đồ án từ lớp giao diện đa nền tảng, tầng xử lý logic và điều hướng cho đến hệ thống lưu trữ, bộ nhớ đệm tầng tốc đều được liên kết cấu trúc chặt chẽ với nhau theo kiến trúc CClient-Server. Việc áp dụng các rào chắn an ninh tiêu chuẩn phối hợp cùng các dịch vụ đám mây và tự động hóa tài chính là bước chuẩn bị cơ sở lý thuyết khoa học đầy đủ, vững chắc để phục vụ cho giai đoạn thiết kế chi tiết hệ thống và hiện thực hóa mã nguồn ở các chương tiếp theo.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 4 tập trung vào quá trình hiện thực hóa các yêu cầu hệ thống và công nghệ đã được phân tích ở các chương trước thành cấu trúc phần mềm cụ thể thông qua các bước thiết kế kiến trúc, thiết kế chi tiết, xây dựng và kiểm thử ứng dụng. Nội dung chương bao gồm việc định hình kiến trúc tổng quan dựa trên mô hình MVC cải tiến, đặc tả chi tiết hệ quản trị cơ sở dữ liệu, giao diện người dùng và luồng xử lý của các lớp chức năng cốt lõi. Đồng thời, chương này cũng trình bày chi tiết về các công cụ phát triển, kết quả thử nghiệm thực tế và mô hình triển khai hệ thống, nhằm minh chứng cho tính đúng đắn và khả năng vận hành ổn định của sản phẩm trong môi trường thực tế.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế và xây dựng dựa trên mẫu kiến trúc Client-Server. Đây là mô hình kiến trúc hiện đại giúp tách biệt hoàn toàn giữa tầng logic xử lý dữ liệu phía máy chủ (Backend) và tầng hiển thị giao diện phía máy trạm (Frontend). Hai thực thể này vận hành độc lập, có vòng đời phát triển riêng biệt và giao tiếp khăng khít với nhau thông qua giao thức chuẩn HTTP/HTTPS bằng cách trao đổi dữ liệu định dạng JSON. Khi áp dụng vào hệ thống thực tế của đề án, mô hình Client-Server được hiện thực hóa và phân rã thành các thành phần kỹ thuật cụ thể.

Thành phần Máy chủ (Backend Server) đóng vai trò là trung tâm xử lý dữ liệu, quản trị nghiệp vụ và cung cấp các dịch vụ API. Backend được xây dựng trên nền tảng framework Django kết hợp với Django REST Framework (DRF), tổ chức thành hai tầng xử lý cốt lõi. Tầng dữ liệu (Database & Storage Layer) được hiện thực hóa dựa trên hệ thống ánh xạ thực thể mối quan hệ Django ORM, kết nối trực tiếp với hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL. Tầng này bao gồm các lớp đối tượng đóng vai trò định nghĩa cấu trúc bảng, mối quan hệ ràng buộc khóa ngoại, và quy định tính toàn vẹn dữ liệu cho bài toán quản lý chung cư. Ngoài ra, thành phần dữ liệu còn được tích hợp thêm bộ nhớ đệm Redis thông qua thư viện django-redis để tối ưu hóa tốc độ truy xuất đối với các yêu cầu API có tần suất đọc cao từ cư dân. Tầng nghiệp vụ và cung cấp dịch vụ (API Provider/Controller Layer) gồm hai thành phần xử lý chính là Serializer và View của DRF. Lớp Serializer chịu trách nhiệm kiểm tra tính hợp lệ của dữ liệu đầu vào từ máy trạm gửi lên, thực hiện quá trình tuần tự hóa và nghịch đảo tuần tự hóa dữ liệu giữa định dạng đối tượng Python và chuỗi JSON. Lớp View đảm nhiệm chức năng tiếp nhận các yêu cầu

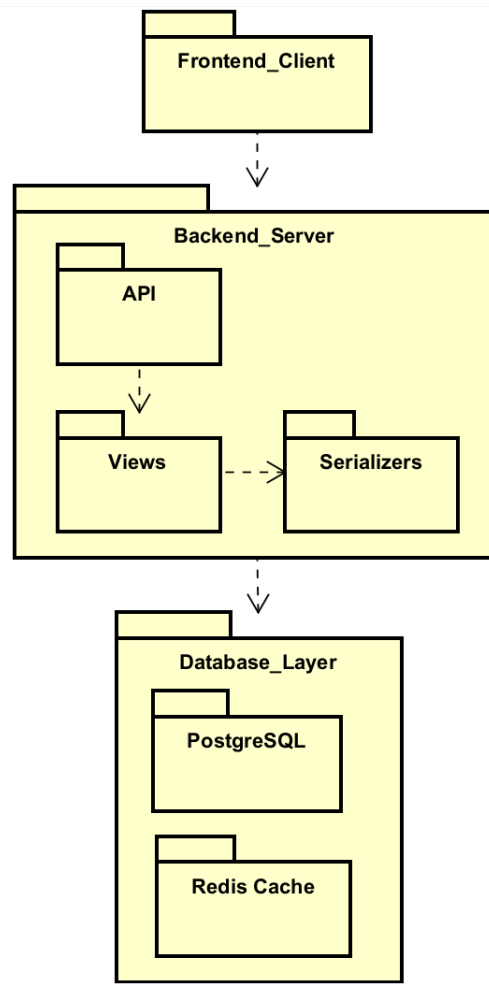
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

HTTP (GET, POST, PUT, DELETE), điều phối luồng xử lý logic nghiệp vụ, thực thi cơ chế xác thực JWT, phân quyền RBAC và đóng gói dữ liệu phản hồi chuẩn hóa để trả về cho Client.

Thành phần Máy trạm (Clients) đóng vai trò là tầng giao diện trực quan (Frontend SPA và Mobile App), đảm nhận nhiệm vụ tương tác trực tiếp với người dùng cuối, thu thập dữ liệu đầu vào và gửi yêu cầu đến Backend. Trong đồ án, thành phần Client được phân rã thành hai phân hệ độc lập. Phân hệ Web Client dành cho cả ban quản trị và cư dân tương tác trên trình duyệt máy tính, được xây dựng dựa trên thư viện React và mô hình Single Page Application. Hệ thống sử dụng React Router để điều hướng trang tức thì và bộ quản lý trạng thái tập trung Redux Toolkit để đồng bộ dữ liệu toàn cục. Giao diện kết nối với API Backend thông qua HTTP Client Axios, tự động đính kèm mã xác thực JWT trong Cookie bảo mật. Phân hệ Mobile Client dành riêng cho đối tượng Cư dân tương tác trên thiết bị di động, được hiện thực hóa qua framework React Native, hệ sinh thái Expo và ngôn ngữ TypeScript. Phân hệ này sử dụng React Navigation để quản lý luồng màn hình vuốt chạm mượt mà, Async Storage để lưu trữ cục bộ các phiên đăng nhập, và Redux Toolkit để đồng bộ trạng thái bất đồng bộ trả về từ API Backend.

4.1.2 Thiết kế tổng quan

Để làm rõ cấu trúc tổ chức của mã nguồn và mối quan hệ giữa các thành phần phần mềm, biểu đồ gói UML được xây dựng dựa trên nguyên tắc phân tầng và cô lập kiến trúc Client-Server. Hệ thống được mô hình hóa thành ba gói chức năng cốt lõi có ranh giới vận hành rõ ràng, bao gồm: Gói Giao diện máy trạm (Client Layer), Gói Dịch vụ điều khiển (Backend API Layer), và Gói Lưu trữ dữ liệu (Database Layer). Việc sắp đặt các cấu trúc này tuân thủ quy tắc thiết kế hệ thống tách biệt, đảm bảo các thành phần đạt tiêu chí liên kết lỏng lẻo và dễ dàng bảo trì độc lập.



Hình 4.1: Biểu đồ phụ thuộc gói tổng quan

Gói Giao diện máy trạm đại diện cho toàn bộ phân hệ Frontend đa nền tảng của hệ thống, bao gồm hai gói con độc lập là ứng dụng Web (React) và ứng dụng di động (React Native). Gói này chịu trách nhiệm thu thập tương tác từ ban quản trị và cư dân, đồng thời thực hiện xử lý hiển thị giao diện động trực tiếp trên thiết bị client. Về mặt kiến trúc, gói Client Layer thiết lập mối quan hệ phụ thuộc hướng trực tiếp đến gói Backend API Layer bên dưới. Điều này có nghĩa là mọi hoạt động thay đổi cấu trúc dữ liệu đầu ra (API Contracts) từ phía máy chủ sẽ ảnh hưởng trực tiếp đến kịch bản xử lý logic và hiển thị dữ liệu của tầng giao diện.

Gói Dịch vụ điều khiển đóng vai trò là hạt nhân xử lý nghiệp vụ trung tâm của hệ thống, được xây dựng trên nền tảng Django REST Framework. Để tinh gọn luồng xử lý, gói này được phân rã thành ba gói chức năng con có mối quan hệ khăng khít với nhau. Gói API đảm nhiệm vai trò cấu hình định tuyến (URLs Routing), tiếp nhận đầu vào của các yêu cầu HTTP từ tầng Client để điều phối đến các phân lớp xử lý tương ứng. Gói Views là nơi tập trung toàn bộ logic nghiệp vụ cốt lõi của bài toán quản lý chung cư, xử lý các bộ lọc an ninh (Xác thực JWT, phân quyền RBAC) và trực tiếp điều phối dữ liệu. Gói Serializers chịu trách nhiệm kiểm tra tính hợp

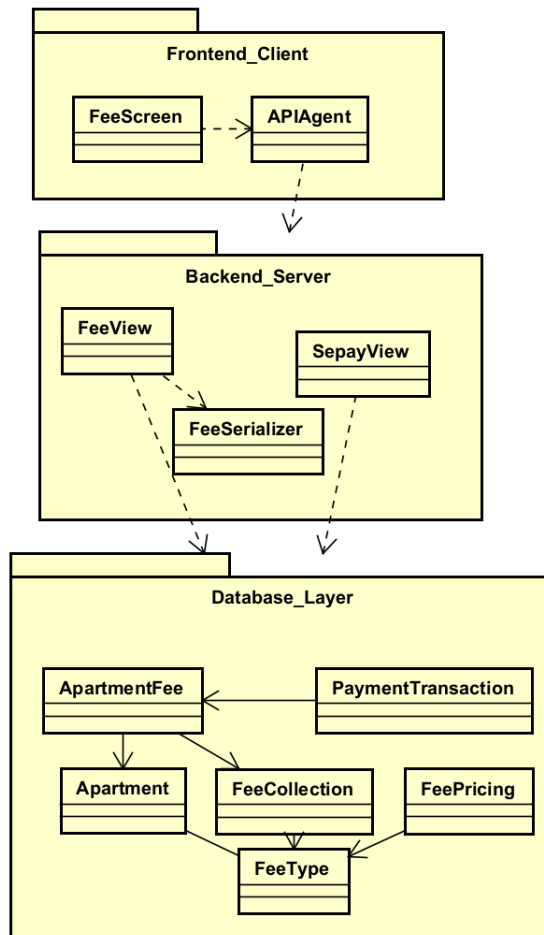
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lệ của dữ liệu và thực hiện quá trình tuần tự hóa dữ liệu cấu trúc thành định dạng chuỗi JSON trước khi phản hồi.

Gói Lưu trữ dữ liệu nằm ở tầng cuối cùng của kiến trúc hệ thống, bao gồm hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL và bộ nhớ đệm phân tán Redis. Gói này chịu trách nhiệm định nghĩa các thực thể thông tin cốt lõi của tòa nhà chung cư, duy trì tính toàn vẹn dữ liệu thông qua các ràng buộc khóa và đảm bảo các giao dịch tài chính tuân thủ nguyên tắc ACID. Gói Lưu trữ dữ liệu vận hành hoàn toàn độc lập, đóng vai trò là kho chứa dữ liệu gốc cho toàn bộ hệ thống và không tồn tại bất kỳ mối liên hệ phụ thuộc ngược nào hướng về các tầng dịch vụ hay giao diện phía trên, giúp triệt tiêu nguy cơ vòng lặp phụ thuộc cấu trúc và nâng cao tính bảo mật dữ liệu.

4.1.3 Thiết kế chi tiết gói

Trong phần này, hệ thống tập trung đặc tả thiết kế chi tiết cấu trúc các lớp thành phần cho nghiệp vụ Quản lý khoản phí. Đây là phân hệ lõi đảm nhiệm toàn bộ chu trình khởi tạo chu kỳ phí, tính toán công nợ căn hộ, và lưu vết giao dịch tài chính. Nhằm đảm bảo tính tường minh theo quy chuẩn tài liệu kỹ thuật, biểu đồ lớp của phân hệ được tổ chức và phân rõ ràng thuộc ba phân lớp kiến trúc Client-Server, đồng thời lược bỏ các thành phần thuộc tính và phương thức nội bộ để tập trung đặc tả các mối quan hệ cấu trúc giữa các lớp.



Hình 4.2: Thiết kế chi tiết gói cho quản lý khoản phí

Tại gói Giao diện (Frontend_Client), các thành phần được thiết kế để xử lý luồng tương tác của cư dân và ban quản trị, đồng thời thu nhận dữ liệu bất đồng bộ từ máy chủ. Lớp FeeScreen đại diện cho giao diện hiển thị danh sách hóa đơn, thông báo công nợ và lịch sử thanh toán trên ứng dụng. Khi người dùng thực hiện thao tác tra cứu hoặc kích hoạt thanh toán, FeeScreen thiết lập mối quan hệ Dependency trở đến lớp tiện ích giao tiếp mạng APIAgent. Lớp APIAgent đóng vai trò đóng gói tham số, thiết lập các yêu cầu gọi HTTP API (gửi kèm mã xác thực) và thiết lập một mũi tên phụ thuộc hướng trực tiếp sang ranh giới của tầng dịch vụ điều khiển phía Backend.

Tại gói Điều khiển nghiệp vụ thuộc Backend_Server, luồng tính toán công nợ và tích hợp cổng đối soát thanh toán tự động được điều phối bởi các bộ điều khiển chuyên biệt của Django REST Framework. Lớp FeeView Tiếp nhận các yêu cầu truy vấn danh sách hoặc khởi tạo khoản phí từ APIAgent. Lớp này thiết lập quan hệ phụ thuộc nét đứt sang lớp tuần tự hóa dữ liệu FeeSerializer để thực hiện quá trình kiểm tra tính hợp lệ của dữ liệu đầu vào và nghịch đảo tuần tự cấu trúc JSON. Đồng thời, FeeView trực tiếp phụ thuộc vào các thực thể dữ liệu ở tầng dưới để

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

thực thi logic tính toán. Lớp SepayView Đóng vai trò là đầu cuối tiếp nhận dữ liệu (Webhook Endpoint) từ kênh trung gian thanh toán SePay. Nhằm mục đích tự động hóa dòng tiền, khi nhận dữ liệu thông báo biến động số dư từ ngân hàng gửi về, SepayView thực thi kiểm tra chữ ký an toàn, thiết lập quan hệ phụ thuộc để tự động khởi tạo bản ghi dữ liệu giao dịch trong lớp PaymentTransaction, và đồng thời truy vấn, cập nhật trực tiếp trạng thái từ chưa thanh toán sang đã thanh toán trên thực thể ApartmentFee.

Tại gói Thực thể dữ liệu, cấu trúc luồng tiền và các định mức biểu phí được quản lý chặt chẽ thông qua các thực thể ánh xạ trực tiếp xuống hệ quản trị cơ sở dữ liệu PostgreSQL bằng cơ chế Django ORM. Mỗi quan hệ giữa các lớp dữ liệu được đặc tả qua quan hệ kết hợp có hướng bằng các mũi tên nét liền dựa trên cấu trúc khóa ngoại. Thực thể trung tâm ApartmentFee (quản lý công nợ chi tiết của từng căn hộ) thiết lập hai mối quan hệ trở trực tiếp về thực thể Apartment và FeeCollection (chu kỳ thu phí). Điều này thể hiện một bản ghi công nợ bắt buộc phải gắn liền với một căn hộ và một đợt thu phí cụ thể. Thực thể FeeCollection (quản lý đợt thu phí theo tháng/năm) thiết lập mối quan hệ kết hợp trở đến thực thể FeeType để xác định danh mục loại phí áp dụng. Thực thể FeePricing (quản lý cấu hình đơn giá) thiết lập mối quan hệ kết hợp hướng về FeeType, cho phép hệ thống tra cứu đơn giá hoặc định mức cố định tương ứng với từng loại phí khi thực hiện thuật toán quét tạo phí tự động. Thực thể FeeType thiết lập quan hệ kết hợp nhiều-nhiều với thực thể Apartment nhằm xác định phạm vi áp dụng của loại phí (áp dụng toàn bộ chung cư hoặc chỉ áp dụng cho một nhóm căn hộ chuyên biệt). Thực thể PaymentTransaction đóng vai trò lưu vết các giao dịch chuyển khoản thành công của cư dân, thiết lập một mối quan hệ kết hợp có hướng trở trực tiếp về thực thể ApartmentFee để phục vụ cho bài toán đối soát tự động và cập nhật trạng thái công nợ của hệ thống quản lý tòa nhà.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Thiết kế giao diện người dùng và trải nghiệm người dùng đóng vai trò quyết định trong việc định hình tính hiệu quả của hệ thống quản lý chung cư. Giao diện được xây dựng nhằm đảm bảo tính nhất quán, tối giản hóa các thao tác xử lý dữ liệu phức tạp của Ban quản trị vận hành trên nền tảng Web và tối ưu hóa sự tiện lợi cho Cư dân khi tương tác trên ứng dụng di động.

Để hệ thống hoạt động ổn định và hiển thị trực quan trên các thiết bị đích, các thông số kỹ thuật về môi trường hiển thị của ứng dụng được thiết lập dựa trên các tiêu chuẩn phân giải phổ biến hiện nay. Giao diện Web được thiết kế đáp ứng hỗ trợ

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

cơ chế cơ giản linh hoạt cho các dải phân giải màn hình khác nhau, sử dụng màu xanh lục bảo làm màu chủ đạo. Giao diện ứng dụng di động dành cho cư dân được thiết kế theo mật độ điểm ảnh của các thiết bị di động thông minh hiện nay với màu sắc chủ đạo là xanh dương. Hệ thống nút bấm được quy chuẩn hóa rõ ràng về hình khối, màu sắc, kích thước. Toàn bộ thông báo hệ thống khi thực hiện tác vụ thành công hoặc thất bại được quy chuẩn xuất hiện bên dưới ở giữa màn hình dưới dạng thẻ thông báo nổi trên web, còn ở ứng dụng di động là các thẻ thông báo hiện ra cho người dùng đọc.

STT	Căn hộ	Tên khoản phí	Tháng	Số tiền	Bắt buộc	Hạn nộp
1	A101	Phí quản lý chung cư	6/2025		Có	
2	A102	Phí quản lý chung cư	6/2025		Có	
3	A103	Phí quản lý chung cư	6/2025		Có	
4	A104	Phí quản lý chung cư	6/2025		Có	
5	A105	Phí quản lý chung cư	6/2025		Có	
6	A101	Phí gửi xe	6/2025		Có	
7	A102	Phí gửi xe	6/2025		Có	
8	A103	Phí gửi xe	6/2025		Có	
9	A101	Ứng hộ lũ lụt	6/2025		Không	

Hình 4.3: Thiết kế giao diện cho màn hình tạo bảng thu phí trên web

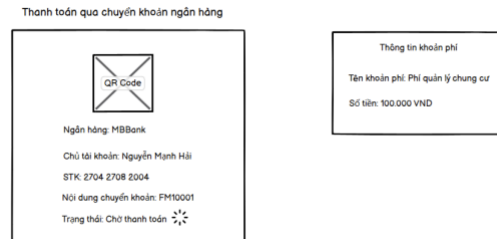
Màn hình tạo bảng thu phí với bảng thu phí hiển thị các cột thông tin bao gồm STT, mã căn hộ, tên khoản phí, chu kỳ tháng năm, số tiền bắt buộc, hạn nộp. Bên cạnh là danh sách các khoản phí chưa được tạo và có thể tạo thêm cho cùng tháng đó.

STT	Căn hộ	Tầng	Diện tích (m2)	Chủ hộ	Email	Trạng thái	Lịch sử cư trú	Hành động
1	A101	1	40	Nguyễn Văn A	nva@gmail.com	Đã bán	👁	+ 🔒 📄
2	A102	1	40	Nguyễn Văn B	nvb@gmail.com	Đã bán	👁	+ 🔒 📄
3	A103	1	35	Nguyễn Văn C	nvc@gmail.com	Đã bán	👁	+ 🔒 📄
4	A104	1	35	Nguyễn Văn D	nvd@gmail.com	Đã bán	👁	+ 🔒 📄
5	A201	2	40			Chưa bán	👁	+ 🔒 📄
6	A202	2	40	Nguyễn Văn B	nvb@gmail.com	Đã bán	👁	+ 🔒 📄
7	A203	2	35	Nguyễn Văn C	nvc@gmail.com	Đã bán	👁	+ 🔒 📄
8	A204	2	35			Chưa bán	👁	+ 🔒 📄
9	A301	3	40	Nguyễn Văn A	nva@gmail.com	Đã bán	👁	+ 🔒 📄
10	A302	3	40	Nguyễn Văn B	nvb@gmail.com	Đã bán	👁	+ 🔒 📄
11	A303	3	35	Nguyễn Văn C	nvc@gmail.com	Đã bán	👁	+ 🔒 📄
12	A304	3	35			Chưa bán	👁	+ 🔒 📄
13	A401	2	40			Chưa bán	👁	+ 🔒 📄

Hình 4.4: Thiết kế giao diện cho màn hình quản lý căn hộ trên web

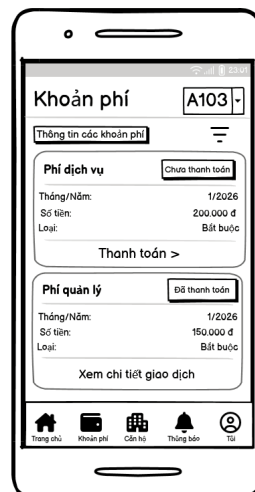
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Màn hình này quản lý toàn bộ thông tin của căn hộ trong tòa nhà. Các trường dữ liệu bao gồm stt, mã căn hộ, tầng, diện tích, họ tên chủ hộ, email, trạng thái căn hộ, lịch sử cư trú được hiển thị. Cột hành động giúp ban quản trị có thể thao tác thêm tài khoản cho căn hộ hoặc vô hiệu hóa tài khoản, sửa thông tin căn hộ. Thanh tìm kiếm giúp tìm kiếm thông tin căn hộ nhanh chóng và tiện lợi.



Hình 4.5: Thiết kế giao diện cho màn hình thanh toán trên web

Màn hình này hỗ trợ cư dân thực hiện giao dịch chuyển khoản nhanh thông qua cơ chế tự động quét mã. Bên trái là khung thông tin chuyển khoản ngân hàng, chứa mã QR Code động được cấu hình tự động, bên dưới ghi rõ tên ngân hàng nhận, chủ tài khoản, số tài khoản, cú pháp nội dung chuyển khoản được định dạng mã hóa đơn. Khối bên phải là hộp thông tin tóm lược chi tiết bao gồm tên khoản phí và chính xác số tiền cần nộp.



Hình 4.6: Thiết kế giao diện cho màn hình khoản phí trên app mobile

Màn hình liệt kê danh sách các loại phí trong tháng, bao gồm các thông tin như trạng thái thanh toán, thông tin số tiền, thời gian thu phí và loại bắt buộc hoặc tự nguyện. Nếu khoản phí chưa được thanh toán sẽ hiển thị nút thanh toán để di chuyển tới màn hình thanh toán, nếu khoản phí đã được thanh toán thì cư dân có thể xem lại khi ấn nút xem chi tiết giao dịch.

4.2.2 Thiết kế lớp

Trong hệ thống quản lý phí chung cư, thiết kế lớp đóng vai trò cốt lõi để đảm bảo tính toàn vẹn dữ liệu và khả năng mở rộng. Phần này tập trung trình bày chi tiết về thiết kế lớp cho nghiệp vụ quản lý khoản phí.

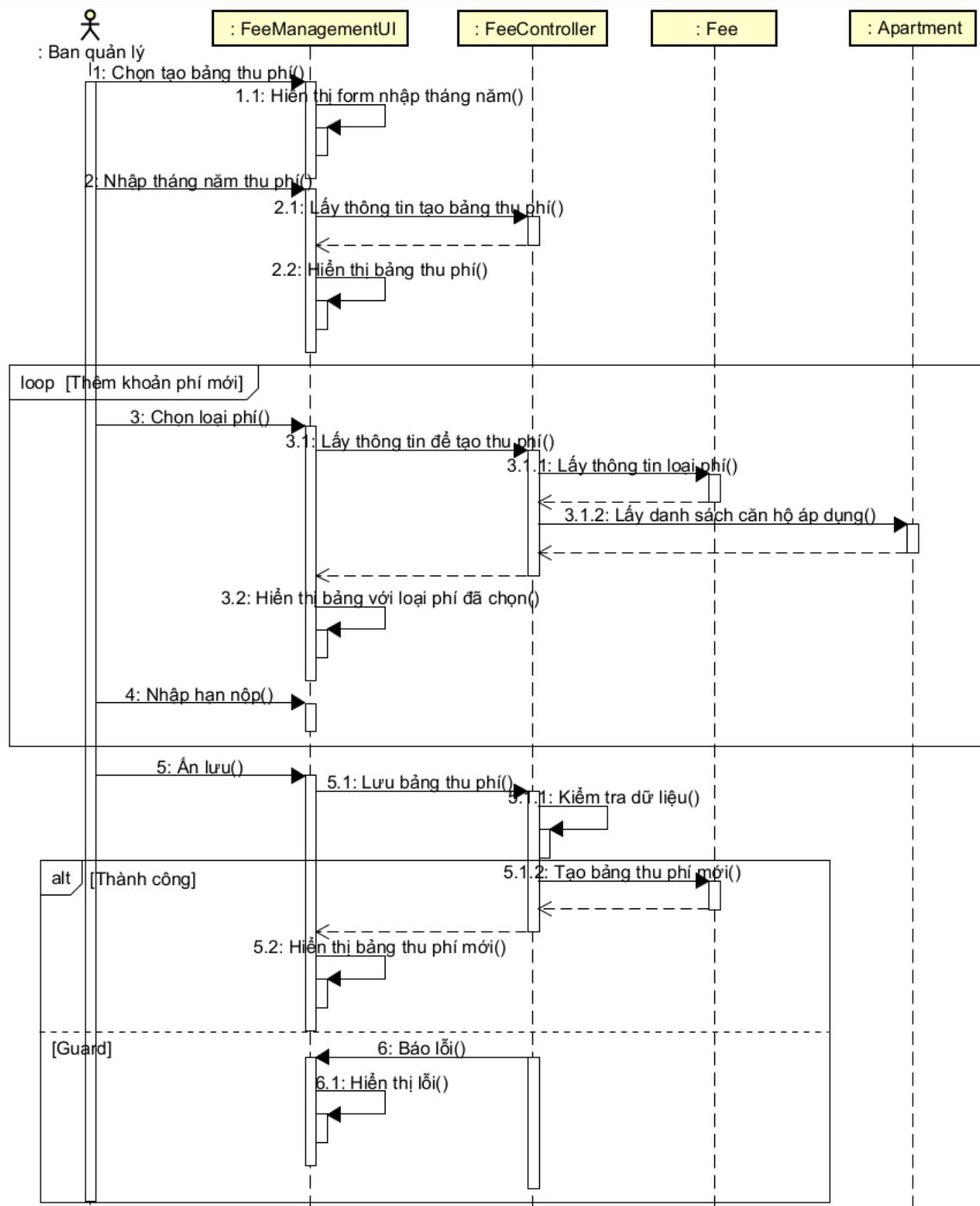
Lớp `FeeType` được thiết kế để định nghĩa cấu hình của các loại phí trong tòa nhà. Lớp này chứa các thuộc tính quan trọng như `typeId` (định danh chính), `feeName` (tên phí), `typeDescription` (mô tả), cùng các thuộc tính trạng thái như `isRequired` (tính bắt buộc), `appliedScope` (phạm vi áp dụng), và `calcMethod` (phương thức tính toán: cố định, diện tích, phương tiện hoặc nhân khẩu).

Tiếp theo, lớp `FeeCollection` đóng vai trò là cầu nối quản lý các đợt thu phí theo thời gian, được xác định bởi các thuộc tính `collectionId`, `month`, `year`, `createdDate` (ngày tạo) và `dueDate` (hạn nộp). Lớp này thiết lập mối quan hệ `ForeignKey` với lớp `FeeType`, cho phép hệ thống quản lý chính xác loại phí nào sẽ được áp dụng cho kỳ thu phí nào.

Lớp `ApartmentFee` là lớp thực thi nghiệp vụ quan trọng nhất, nơi lưu trữ thông tin về nghĩa vụ tài chính của từng căn hộ trong một kỳ thu phí cụ thể. Lớp này bao gồm các thuộc tính như `apartmentFeeId`, `amount` (số tiền), `status` (trạng thái thanh toán: `paid`, `unpaid`, `deleted`), đồng thời duy trì mối quan hệ liên kết trực tiếp với lớp `Apartment` và `FeeCollection`. Sự phân tách này cho phép hệ thống theo dõi chi tiết tình trạng công nợ của từng căn hộ một cách minh bạch.

Lớp `PaymentTransaction` đảm bảo tính xác thực cho các giao dịch tài chính thông qua các thuộc tính `transactionId` (mã giao dịch từ cổng thanh toán), `apartmentFee` (liên kết với hóa đơn), `amount` (số tiền thanh toán), `paymentDate` (ngày thực hiện) và `status` (trạng thái thành công hoặc thất bại). Lớp này không chỉ lưu trữ lịch sử giao dịch mà còn là cơ sở để hệ thống tự động đối soát và cập nhật trạng thái của lớp `ApartmentFee`. Việc thiết kế các thuộc tính và mối quan hệ chặt chẽ giữa các lớp này giúp hệ thống đạt được sự nhất quán cao trong xử lý nghiệp vụ, từ lúc khởi tạo bảng phí đến khi ghi nhận thành công giao dịch của cư dân.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

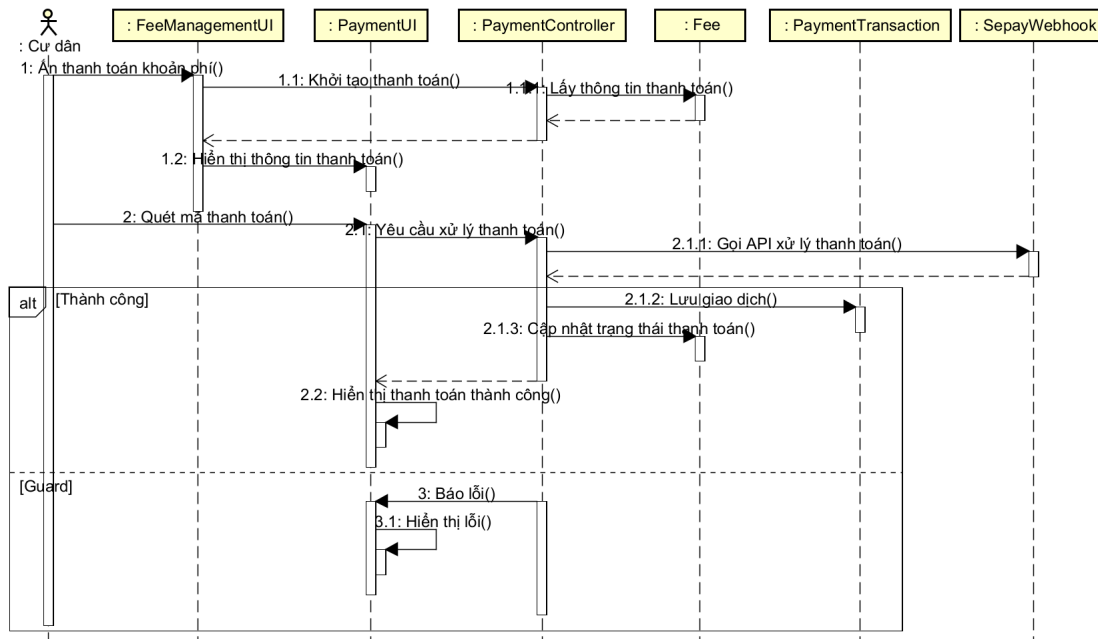


Hình 4.7: Biểu đồ trình tự cho usecase Tạo bảng thu phí

Biểu đồ này mô tả cách Ban quản lý khởi tạo một kỳ thu phí mới cho toàn bộ chung cư. Khi Ban quản trị thực hiện yêu cầu "Tạo bảng thu phí" thông qua FeeManagementUI, hệ thống sẽ khởi tạo một form nhập liệu cho tháng và năm cần thu. Sau khi các thông số này được gửi đi, FeeController sẽ bắt đầu chuỗi logic nghiệp vụ bằng việc tương tác với Fee để lấy thông tin các loại phí đang hoạt động. Tiếp theo, hệ thống tự động truy vấn danh sách căn hộ từ đối tượng Apartment để thiết lập đối tượng áp dụng cho bảng phí. Trong vòng lặp, hệ thống sẽ tính toán số tiền dựa trên phương thức đã định nghĩa cho từng loại phí và căn hộ, sau đó lưu trữ

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

kết quả vào cơ sở dữ liệu. Nếu dữ liệu không hợp lệ, hệ thống sẽ trả về thông báo lỗi để quản trị viên sửa đổi. Cuối cùng, khi dữ liệu đã được xác nhận chính xác, hệ thống sẽ tạo mới bảng thu phí và hiển thị danh sách hoàn chỉnh cho người dùng.



Hình 4.8: Biểu đồ trình tự cho usecase Thanh toán

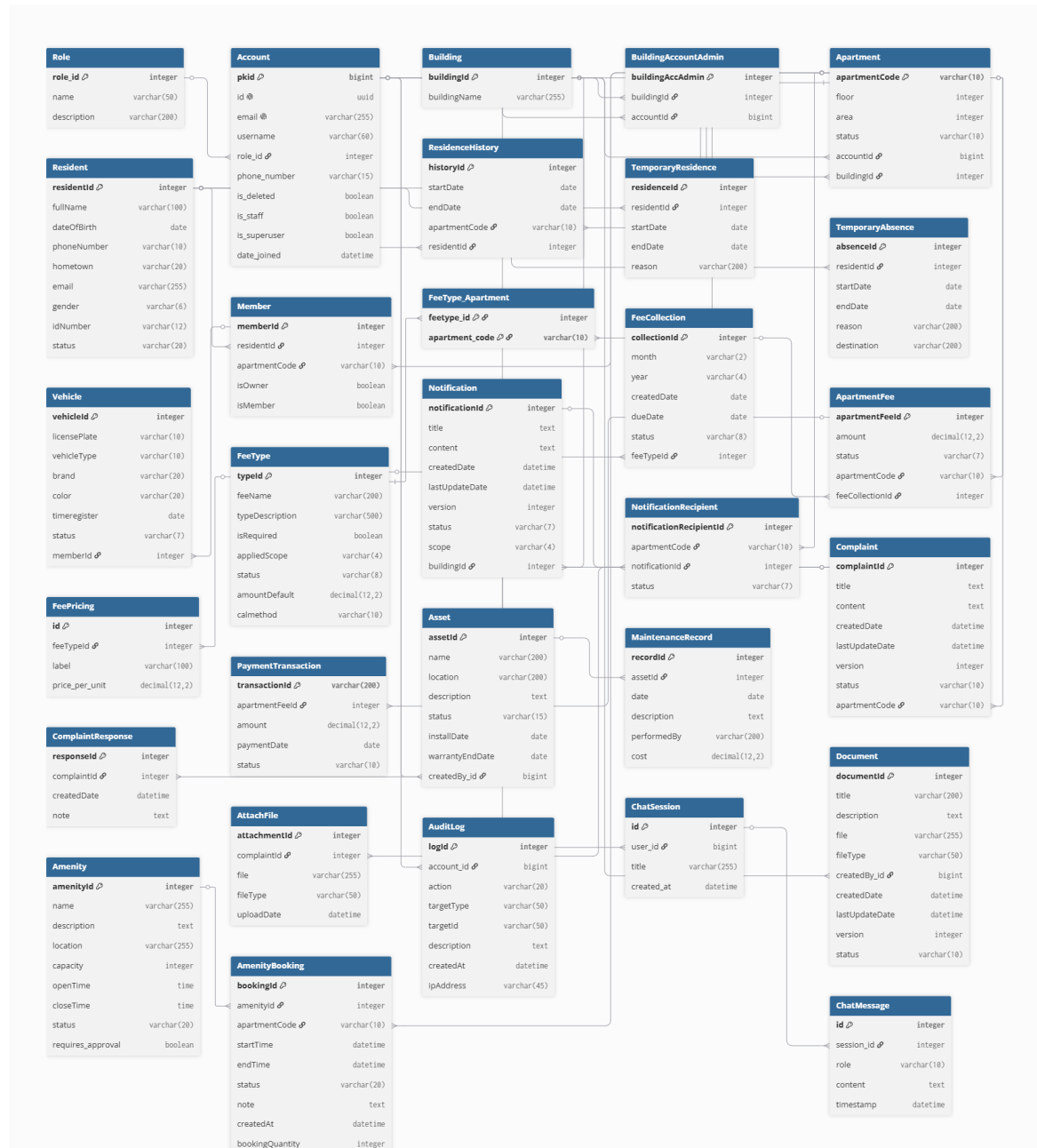
Biểu đồ này mô tả trải nghiệm của cư dân khi thực hiện nghĩa vụ tài chính thông qua ứng dụng. Khi cư dân truy cập vào giao diện quản lý khoản phí và chọn "Tiến hành thanh toán", PaymentUI sẽ gửi yêu cầu tới PaymentController để thực hiện thủ tục khởi tạo giao dịch. Hệ thống sẽ kết nối với BankGateway để thực hiện giao dịch thanh toán thực tế. Sau khi cổng thanh toán trả về kết quả, biểu đồ trình tự phân nhánh theo hai trường hợp: Nếu thanh toán thành công, đối tượng PaymentTransaction sẽ ghi nhận lại giao dịch và PaymentController sẽ cập nhật trạng thái của hóa đơn từ "chưa thanh toán" sang "đã thanh toán" trong hệ thống, đồng thời thông báo xác nhận thành công cho cư dân. Ngược lại, nếu giao dịch thất bại, hệ thống sẽ báo lỗi cho cư dân và hiển thị tùy chọn cho phép cư dân thực hiện lại lệnh thanh toán. Luồng này đảm bảo rằng trạng thái của hóa đơn luôn đồng bộ với kết quả phản hồi từ ngân hàng.

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống quản lý chung cư yêu cầu một hạ tầng dữ liệu ổn định, đảm bảo tính toàn vẹn và khả năng truy vấn phức tạp cho các báo cáo tài chính. Hệ thống lựa chọn PostgreSQL – một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ – làm nền tảng lưu trữ. PostgreSQL được ưu tiên nhờ khả năng xử lý các giao dịch ACID, đặc biệt quan trọng trong các nghiệp vụ liên quan đến tiền tệ và thông tin cư dân.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Sơ đồ dưới đây trình bày mô hình logic của hệ thống, thể hiện các thực thể quan trọng nhất và mối quan hệ giữa chúng. Biểu đồ này tập trung vào luồng nghiệp vụ thay vì các chi tiết kỹ thuật đơn thuần, giúp làm rõ mối liên kết giữa nhóm người dùng, căn hộ, tài chính và các dịch vụ tiện ích.



Hình 4.9: Biểu đồ thực thể liên kết

Bảng Role định nghĩa các phân quyền, trong khi bảng Account lưu trữ thông tin xác thực cho tất cả người dùng. Đây là lớp bảo vệ đầu tiên, đảm bảo mọi thao tác trên hệ thống đều được định danh và phân quyền chính xác.

Bảng Building là gốc, kết nối với bảng Apartment theo quan hệ một tòa nhà bao gồm nhiều căn hộ. Bảng BuildingAccountAdmin đóng vai trò bảng trung gian,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

kết nối Tài khoản quản trị với tòa nhà tương ứng, cho phép phân cấp quản lý nếu hệ thống vận hành nhiều tòa nhà cùng lúc. Hệ thống này giúp định nghĩa rõ ràng không gian sống mà các nghiệp vụ khác sẽ dựa vào để vận hành.

Bảng Resident lưu thông tin cá nhân, còn Member đóng vai trò bảng liên kết giữa Resident và Apartment, cho phép một người có thể đăng ký ở nhiều căn hộ hoặc một căn hộ có nhiều thành viên. Các bảng ResidenceHistory, TemporaryResidence, và TemporaryAbsence được kết nối với Resident, đóng vai trò là nhật ký ghi vết, giúp hệ thống theo dõi chính xác lịch sử cư trú và tạm vắng của cư dân tại bất kỳ thời điểm nào.

Bảng FeeType định nghĩa các loại phí và quy tắc tính tiền (FeePricing), tạo tiền đề để bảng FeeCollection phát hành các đợt thu phí theo thời gian. Mỗi quan hệ giữa ApartmentFee và các bảng còn lại là trung tâm: nó kế thừa thông tin từ FeeCollection và Apartment, sau đó ghi nhận trạng thái thanh toán thông qua bảng PaymentTransaction. Mỗi quan hệ từ ApartmentFee tới PaymentTransaction cho phép hệ thống lưu lại đầy đủ lịch sử giao dịch thanh toán cho mỗi hóa đơn.

Bảng Notification và Complaint đều được gắn kết với Apartment để xác định đối tượng nhận hoặc đối tượng khiếu nại. Bảng Asset quản lý cơ sở vật chất, kết nối với MaintenanceRecord để theo dõi lịch sử bảo trì. Đặc biệt, bảng AuditLog đóng vai trò là "hộp đen" ghi lại mọi thao tác của người dùng trong hệ thống, liên kết trực tiếp với bảng Account để đảm bảo tính minh bạch và trách nhiệm giải trình.

Bảng Amenity đóng vai trò là danh mục các tiện ích khả dụng. Mỗi quan hệ giữa Amenity và AmenityBooking cho phép hệ thống ghi nhận lịch sử đặt chỗ cho từng tiện ích. Bảng AmenityBooking đồng thời kết nối với Apartment, giúp xác định căn hộ nào đã thực hiện yêu cầu đặt chỗ. Cấu trúc này không chỉ giúp quản lý lịch trình sử dụng tài sản chung một cách khoa học mà còn ngăn chặn tình trạng chồng chéo thời gian thông qua các kiểm tra logic từ startTime và endTime.

Bảng ChatSession lưu trữ phiên hội thoại của một Account, liên kết với bảng ChatMessage. Thiết kế này cho phép hệ thống lưu vết lịch sử trò chuyện giữa cư dân và Ban quản lý hoặc Chatbot hỗ trợ.

Bảng Document được quản lý bởi Account, phục vụ lưu trữ các thông báo, quy định hoặc văn bản pháp lý. Mỗi tài liệu đều được định danh phiên bản và trạng thái, giúp đảm bảo cư dân luôn tiếp cận được thông tin cập nhật mới nhất.

Việc phân tách và thiết lập các mối quan hệ chặt chẽ giữa các nhóm bảng này không chỉ giúp dữ liệu không bị dư thừa mà còn cho phép hệ thống thực hiện các truy vấn phức tạp, chẳng hạn như: tính tổng doanh thu theo căn hộ, tra cứu lịch sử

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

cư trú của một người, hay báo cáo tình trạng bảo trì tài sản trong tòa nhà một cách nhanh chóng và chính xác.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Hệ thống đã sử dụng các cộng cụ và thư viện như trong bảng dưới để có thể hoàn thiện các yêu cầu về mặt nghiệp vụ.

Mục đích	Công cụ (Phiên bản)	Địa chỉ URL
Backend Framework	Django 5.2.8	https://www.djangoproject.com/
API Development	Django REST Framework 3.16.1	https://www.django-rest-framework.org/
Web	React 19.2.0	https://react.dev/
Mobile App	Expo 54.0.33 / React Native 0.81.5	https://expo.dev/ https://reactnative.dev/
State Management	Redux Toolkit 2.9.1	https://redux-toolkit.js.org/
Task Queue	Celery 5.4.0	https://docs.celeryq.dev/
Cache	Redis 7.1.0	https://redis.io/
Media Storage	Cloudinary 1.44.2	https://cloudinary.com/
AI Integration	Google GenAI 1.74.0	https://aistudio.google.com/
Database	PostgreSQL	https://www.postgresql.org/
IDE Lập trình	Visual Studio Code	https://code.visualstudio.com/

Bảng 4.1: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Trải qua quá trình nghiên cứu lý thuyết, thiết kế kiến trúc và triển khai kỹ thuật thực tế, đề tài đã hoàn thành trọn vẹn các mục tiêu đặt ra, đóng gói thành công các sản phẩm phần mềm hoàn chỉnh phục vụ bài toán quản lý vận hành chung cư. Toàn bộ hệ thống được xây dựng dựa trên kiến trúc phân tách hoàn toàn giữa tầng dịch vụ và tầng giao diện hiển thị, đảm bảo tính bao đóng, tính bảo mật cao và khả năng chịu tải ổn định. Các sản phẩm đóng gói cuối cùng của đề án được cấu trúc thành các thành phần cốt lõi liên kết chặt chẽ với nhau bao gồm hệ thống quản trị trung tâm dạng mã nguồn Web cho Ban quản lý, ứng dụng di động đa nền tảng cho cư dân, và hệ thống dịch vụ lõi API xử lý logic nghiệp vụ cùng các tác vụ ngầm.

Phân hệ Web Dashboard cung cấp công cụ điều hành tập trung toàn bộ chuỗi hoạt động của chung cư. Vai trò cốt lõi của thành phần này thể hiện qua việc quản lý thông tin cư dân, theo dõi phương tiện, phê duyệt các yêu cầu phản ánh, cho đến việc tự động hóa tính toán và khởi tạo hóa đơn tài chính hàng tháng. Ý nghĩa thực tiễn của phân hệ Web là chuẩn hóa và số hóa toàn bộ quy trình làm việc vốn trước

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

đây phụ thuộc vào giấy tờ hoặc các bảng tính thủ công rời rạc, giúp tối ưu hóa hiệu suất làm việc của bộ máy quản lý.

Ứng dụng di động đa nền tảng dành cho đối tượng cư dân đóng vai trò là cầu nối giao tiếp trực tiếp theo thời gian thực giữa người dân và ban quản lý. Ý nghĩa cốt lõi của ứng dụng di động là nâng cao chất lượng cuộc sống của cư dân thông qua việc tiếp nhận thông báo tức thời, chủ động tra cứu lịch sử, quét mã QR thanh toán hóa đơn tự động dưới sự hỗ trợ của kỹ thuật quét trạng thái chu kỳ ngắn, cũng như gửi các kiến nghị sửa chữa kèm hình ảnh trực quan mà không cần gặp mặt trực tiếp.

Chịu trách nhiệm vận hành toàn bộ luồng dữ liệu của hai phân hệ giao diện trên là Hệ thống dịch vụ lõi Backend API và các tác vụ xử lý ngầm. Đây là bộ não trung tâm chịu trách nhiệm thực thi các thuật toán phân quyền đa lớp, xử lý bất đồng bộ dữ liệu hóa đơn thông qua cổng kiểm tra chữ ký số Webhook, quản lý hàng đợi tác vụ gửi email thông báo tự động và duy trì tính toàn vẹn dữ liệu cho toàn bộ hệ thống lưu trữ thông tin chung cư.

Các thông số chi tiết của hệ thống được tổng hợp và thống kê trong bảng dưới đây. Dựa trên các số liệu thống kê thực tế, tổng khối lượng mã nguồn thô đạt xấp xỉ 30500 dòng code, trải dài trên 207 file mã nguồn.

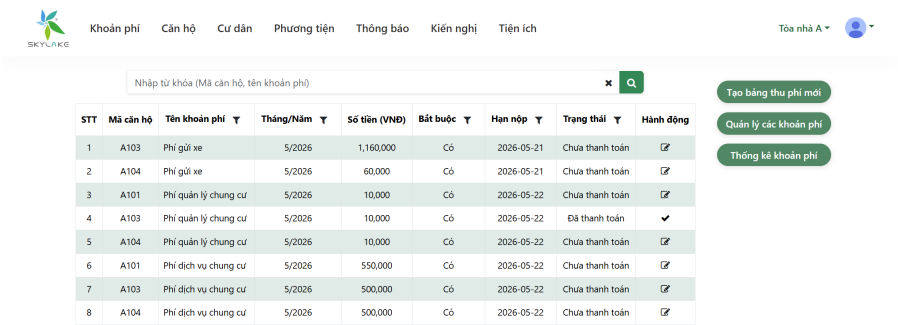
Thành phần	Công nghệ	Số file	Số dòng Code	Dung lượng
Backend	DRF	40	≈ 4600	13 MB
Giao diện Web	React JS	93	≈ 12400	8 MB
Ứng dụng di động	React Native	74	≈ 13500	90 MB

Bảng 4.2: Thống kê quy mô mã nguồn hệ thống

4.3.3 Minh họa các chức năng chính

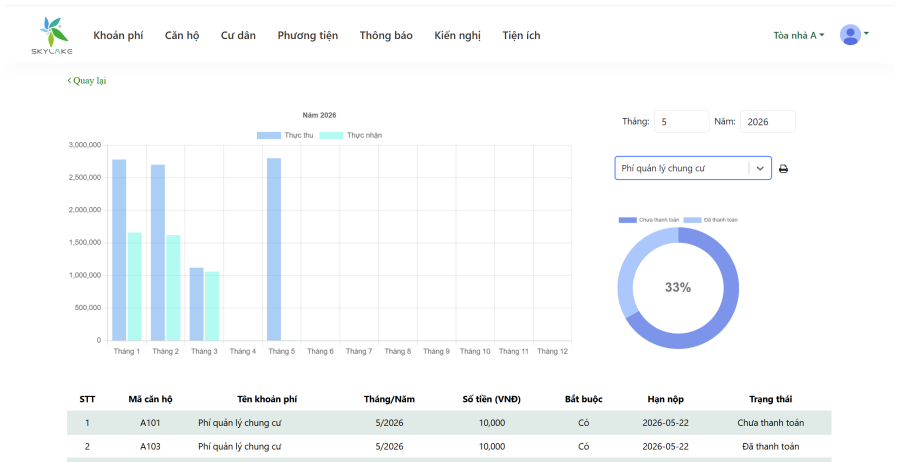
Dưới đây là một số hình ảnh minh họa hệ thống trên thực tế ở cả giao diện web và ứng dụng di động.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



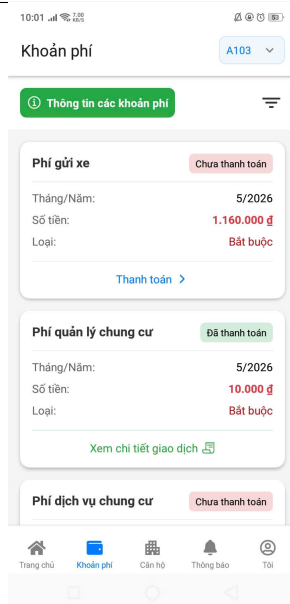
Hình 4.10: Màn hình quản lý khoản phí cho ban quản trị

Giao diện quản lý khoản phí của ứng dụng Skylake được thiết kế trực quan với tông màu xanh lá chủ đạo, bao gồm thanh điều hướng chức năng ở phía trên, ô tìm kiếm nhanh và bảng dữ liệu chi tiết hiển thị trạng thái thanh toán của từng căn hộ. Hệ thống hỗ trợ người quản lý vận hành hiệu quả thông qua việc sắp xếp khoa học các cột thông tin (từ mã căn hộ đến hạn nộp) cùng cụm nút chức năng bên phải, cho phép thực hiện nhanh các tác vụ tạo bảng thu phí, quản lý danh mục và xem thống kê tổng quan.



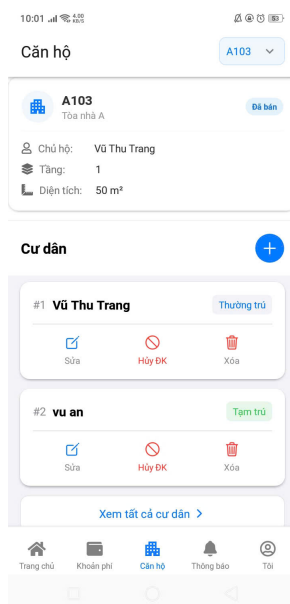
Hình 4.11: Màn hình thống kê khoản phí cho ban quản trị

Giao diện thống kê khoản phí của Skylake mang đến cái nhìn trực quan về tình hình tài chính tòa nhà với biểu đồ cột so sánh giữa "Thực thu" và "Thực nhận" theo từng tháng, kết hợp cùng biểu đồ tròn thể hiện tỷ lệ phần trăm các khoản đã và chưa thanh toán. Người dùng có thể dễ dàng tùy chỉnh dữ liệu hiển thị thông qua các bộ lọc theo tháng, năm và loại phí, đồng thời hệ thống cung cấp danh sách chi tiết các khoản phí liên quan phía dưới để quản trị viên nắm bắt nhanh chóng tình trạng đóng phí của từng căn hộ trong thời gian thực.



Hình 4.12: Màn hình khoản phí cho cư dân

Màn hình này được thiết kế theo dạng danh sách thẻ giúp cư dân dễ dàng theo dõi tình hình tài chính cá nhân. Mỗi loại phí (phí gửi xe, phí quản lý, phí dịch vụ) được hiển thị trong một khung riêng biệt, nêu rõ thông tin về thời gian, số tiền và tính chất (bắt buộc). Điểm nhấn của giao diện là nhãn trạng thái trực quan, đi kèm với nút hành động "Thanh toán" hoặc "Xem chi tiết giao dịch" giúp người dùng thao tác nhanh chóng ngay trên thiết bị di động.



Hình 4.13: Màn hình thông tin về căn hộ cho cư dân

Giao diện này cung cấp cái nhìn tổng quan về thông tin bất động sản và danh sách nhân khẩu cư trú. Phần trên hiển thị thông số căn hộ. Phần dưới quản lý cư

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

dân được trình bày mạch lạc, trong đó mỗi cư dân là một thể thông tin kèm theo các nút chức năng tiện ích như "Sửa", "Hủy ĐK", và "Xóa". Bộ cục này giúp chủ hộ hoặc người quản lý dễ dàng kiểm soát nhân khẩu (thường trú/tạm trú) trong căn hộ thông qua các thao tác chạm đơn giản.

4.4 Kiểm thử

4.4.1 Phương pháp và kỹ thuật kiểm thử áp dụng

Đồ án tập trung áp dụng kỹ thuật kiểm thử hộp đen để đánh giá toàn diện các chức năng phần mềm dựa trên yêu cầu đặc tả nghiệp vụ, đồng thời tích hợp phương pháp kiểm thử tích hợp hệ thống nhằm xác thực tính chính xác của luồng dữ liệu truyền tải giữa các phân hệ Backend API, Web Dashboard và Ứng dụng di động. Giải thuật phân vùng tương đương được sử dụng làm nền tảng cốt lõi để xây dựng các kịch bản kiểm thử. Kỹ thuật phân vùng tương đương cho phép chia dải dữ liệu đầu vào của cư dân thành các vùng đại diện hợp thức và bất hợp thức, từ đó giảm thiểu số lượng ca kiểm thử cần thiết nhưng vẫn bao phủ tối đa các tình huống có thể xảy ra.

4.4.2 Kịch bản kiểm thử chức năng Tạo bảng thu phí tự động

Chức năng khởi tạo bảng thu phí dịch vụ đóng vai trò quyết định tính chính xác và minh bạch trong công tác điều hành tài chính của Ban quản lý tòa nhà. Kịch bản kiểm thử được thiết kế dựa trên quy trình xử lý luồng dữ liệu của lớp dịch vụ, mô phỏng hành vi khi người dùng có quyền Quản trị viên thực hiện gửi yêu cầu cấu hình tháng, năm cùng danh sách các loại phí đi kèm mốc thời gian hạn nộp cụ thể lên hệ thống. Ngay sau khi tiếp nhận dữ liệu đầu vào, máy chủ API sẽ thực hiện chuyển đổi chuỗi ký tự hạn nộp sang định dạng ngày tháng tiêu chuẩn, kiểm tra điều kiện ràng buộc mốc thời gian so với ngày hiện tại để ngăn chặn các sai sót nhập liệu. Khi dữ liệu thời gian vượt qua kiểm thử logic, hệ thống tự động bóc tách các phương thức tính toán được định nghĩa sẵn trong cơ sở dữ liệu để áp dụng đơn giá tương ứng cho từng căn hộ. Các thuật toán này bao gồm việc nhân đơn giá với diện tích thông thủy đối với phí quản lý, đếm số lượng phương tiện đang hoạt động để tính phí trông giữ xe, hoặc áp dụng mức thu cố định cho các dịch vụ tiện ích đồng bộ. Toàn bộ tiến trình tính toán dữ liệu của hàng ngàn căn hộ, khởi tạo hóa đơn chưa thanh toán, tạo lập thông báo chung của tòa nhà và phân phối danh sách người nhận thông báo đều được bao đóng trong một giao dịch an toàn duy nhất để đảm bảo tính toàn vẹn tuyệt đối. Các ca kiểm thử được thiết lập chặt chẽ nhằm kiểm tra tính ổn định của toàn bộ chu trình nghiệp vụ phức tạp này.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

No	Mục tiêu	Dữ liệu đầu vào	Kết quả kỳ vọng	Trạng thái
TC01	Luồng tạo bảng phí với ngày hợp lệ	Điền hạn nộp dạng chuỗi đúng quy chuẩn	Hệ thống tạo bảng phí đúng số tiền và gửi thông báo nộp phí cho cư dân	Đạt
TC02	Kiểm tra định dạng hạn nộp sai quy cách	Nhập hạn nộp không tuân thủ định dạng	Hệ thống chặn, phản hồi yêu cầu sai định dạng	Đạt
TC03	Kiểm tra hạn nộp nhỏ hơn ngày hiện tại	Chọn hạn nộp trùng hoặc trước ngày hôm nay	Hệ thống báo lỗi mốc thời gian không hợp lệ	Đạt
TC04	Xác thực phân quyền thực thi API	Sử dụng tài khoản cư dân để gọi lệnh tạo	Hệ thống từ chối truy cập	Đạt

Bảng 4.3: Kịch bản kiểm thử chi tiết chức năng tạo bảng thu phí

Trong quá trình thực thi các ca kiểm thử trên, hệ thống chứng minh sự ổn định cao nhờ cơ chế bắt lỗi tầng lọc dữ liệu của Django Rest Framework. Luồng dữ liệu chạy qua tầng kiểm thử thực tế cho thấy các đối tượng lưu trữ giá trị phí được khởi tạo đúng tuần tự ngay sau khi người dùng tương tác với nút lưu, các phép tính toán nhân với số tháng chu kỳ biên hoặc xử lý công nợ lũy kế đều trả về kết quả khớp hoàn toàn với kịch bản kiểm thử thủ công trên bảng tính đối chiếu. Hệ thống bảo bọc được tính toán vẹn và không xảy ra hiện tượng sai lệch hay mất mát dữ liệu khi ghi nhận vào cơ sở dữ liệu.

4.4.3 Kịch bản kiểm thử chức năng thanh toán hóa đơn

Chức năng thanh toán hóa đơn kết hợp kỹ thuật Short Polling là chức năng có độ phức tạp cao nhất, đòi hỏi sự tương tác liên tục giữa ứng dụng di động của cư dân, máy chủ dịch vụ và cổng thanh toán. Kịch bản kiểm thử cho chức năng này tập trung vào việc mô phỏng các biến động số dư tài khoản ngân hàng, xử lý kiểm tra tính hợp pháp của chữ ký bảo mật, bẫy các lỗi chia cho không hoặc ngoại lệ lỗi toán học trong quá trình phân chia tỷ lệ dòng tiền, và kiểm tra tính năng tự động hủy bỏ bộ định thời quét trạng thái trên thiết bị di động.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

No	Mục tiêu	Dữ liệu đầu vào	Kết quả kỳ vọng	Trạng thái
TC01	Giao dịch thành công	Giao dịch đúng số tiền và nội dung	App chuyển màn hình thành công, lưu lịch sử giao dịch	Đạt
TC02	Xử lý sai lệch số tiền thanh toán	Số tiền chuyển khoản khác trên hóa đơn	Hệ thống không tìm thấy bản ghi trùng khớp, trả về mã lỗi 404	Đạt
TC03	Nội dung chuyển khoản sai	Nội dung thông tin giao dịch trống hoặc sai cú pháp mã phí	Hệ thống chặn, phản hồi lỗi không tìm thấy định danh khoản phí	Đạt
TC04	Cư dân chủ động thoát màn hình thanh toán	Nhấn nút quay lại khi ở màn hình thanh toán	Polling lập tức bị hủy	Đạt
TC05	Thanh toán trùng cho hóa đơn đã xử lý	Thanh toán lại một hóa đơn đã thanh toán trước đó	Hệ thống chặn do hóa đơn không còn ở trạng thái unpaid, trả lỗi 404	Đạt

Bảng 4.4: Kịch bản kiểm thử chi tiết chức năng Thanh toán hóa đơn

Kết quả thực nghiệm cho thấy phân hệ thanh toán vận hành mượt mà nhờ cơ chế đồng bộ hóa bất đồng bộ thông minh, đảm bảo rằng khi gặp các phép tính lỗi hoặc dữ liệu phản hồi trống từ bên thứ ba, hệ thống không rơi vào trạng thái treo kết nối mà lập tức kích hoạt luồng trả về thông báo lỗi an toàn cho người dùng di động. Việc giải phóng bộ nhớ của vòng lặp Short Polling trên môi trường React Native hoạt động chính xác theo vòng đời component, triệt tiêu hoàn toàn hiện tượng rò rỉ tài nguyên mạng trên thiết bị di động của cư dân.

4.4.4 Tổng kết và phân tích kết quả kiểm thử

Sau khi hoàn thành toàn bộ các kịch bản kiểm thử được thiết kế cho các chức năng cốt lõi của hệ thống quản lý chung cư, số liệu thống kê thực tế ghi nhận tổng cộng tám ca kiểm thử đã được triển khai trên cả nền tảng Web và Ứng dụng di động. Kết quả thu được đạt tỷ lệ thành công tuyệt đối khi toàn bộ các ca kiểm thử đều vượt qua các tiêu chí đánh giá và không ghi nhận bất kỳ lỗi nghiêm trọng nào liên quan đến tầng logic nghiệp vụ hay thất thoát cơ sở dữ liệu.

4.5 Triển khai

Hệ thống quản lý vận hành chung cư thông minh được định cấu hình và triển khai trên môi trường điện toán đám mây theo mô hình phân tán đa nền tảng, nhằm tối ưu hóa chi phí vận hành và tận dụng tối đa thế mạnh của từng nhà cung cấp dịch

vụ hạ tầng chuyên biệt.

Thành phần	Vai trò trong kiến trúc	Hạ tầng triển khai
Backend API	Xử lý logic nghiệp vụ trung tâm	Render Cloud
Database	Lưu trữ toàn vẹn thông tin chung cư	Render PostgreSQL
Web	Giao diện quản trị tập trung	Vercel Edge Server
Mobile App	Giao diện cho cư dân	Tệp tin cài đặt APK
Cache	Hàng đợi thông báo và bộ nhớ đệm	Upstash Redis
Celery Worker	Xử lý ngầm lập lịch tự động	Railway Platform

Bảng 4.5: Hạ tầng triển khai các thành phần hệ thống

Tầng xử lý trung tâm bao gồm dịch vụ lõi Backend và hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL được cấu hình đồng bộ trên nền tảng Render tại khu vực máy chủ Singapore nhằm giảm thiểu tối đa độ trễ mạng khi tương tác dữ liệu.

Giao diện Web được biên dịch tối ưu hóa và xuất bản trên hạ tầng Vercel, giúp đẩy nhanh tốc độ tải trang tĩnh thông qua mạng lưới phân phối nội dung toàn cầu.

Ứng dụng di động được đóng gói thành tệp tin APK qua nền tảng trung gian Expo Application Services, giúp tự động tối ưu hóa tài nguyên giao diện và nén toàn bộ nhân chạy ngầm vào một tệp tin duy nhất. Trong tiến trình thực nghiệm, hệ thống gặp khó khăn khi triển khai ứng dụng lên chợ trực tuyến Google Play. Các rào cản này xuất phát từ chính sách kiểm duyệt nghiêm ngặt đối với ứng dụng nội bộ, đòi hỏi yêu cầu khắt khe về việc duy trì tài khoản kiểm thử tối thiểu hai mươi người trong vòng mười bốn ngày liên tục. Nhằm khắc phục khó khăn về thủ tục hành chính và kịp thời tối ưu hóa thời gian, cấu hình đóng gói đã được chuyển dịch sang cơ chế tạo lập tệp tin phân phối dưới dạng tệp tin APK cài đặt trực tiếp.

Hệ thống hàng đợi tin nhắn và bộ nhớ đệm hiệu năng cao Redis được vận hành độc lập trên nền tảng Upstash theo mô hình phi máy chủ nhằm duy trì kết nối phi ngừng, đóng vai trò làm trung gian truyền tải tác vụ.

Phân hệ quản lý và thực thi các tác vụ ngầm Celery ban đầu được định hướng triển khai thành một dịch vụ container độc lập trên nền tảng đám mây Railway. Tuy nhiên, quá trình vận hành thực tế đã ghi nhận lỗi ngoại lệ nghiêm trọng tại tầng liên kết mạng giữa Railway và hệ quản trị cơ sở dữ liệu PostgreSQL trên Render. Hệ thống liên tục phản hồi lỗi ngắt kết nối lớp bảo mật SSL đột ngột khi thực thi tác vụ quét cấu hình phí gửi đến cư dân. Qua phân tích kỹ thuật, nguyên nhân xuất phát từ sự xung đột chính sách an toàn thông tin liên nền tảng; trong đó Render chủ

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

động đóng phiên kết nối ngoại vi có thời gian duy trì quá lâu nhằm bảo vệ cơ sở dữ liệu, gây nghẽn luồng xử lý bất đồng bộ của Celery. Từ vấn đề đó nên hệ thống lựa chọn loại bỏ việc áp dụng Celery Worker trên môi trường triển khai thực tế giúp giảm rủi ro rò rỉ kết nối dữ liệu liên nền tảng, bảo bọc tính tối giản, an toàn và duy trì luồng vận hành thông suốt cho toàn bộ hệ thống quản lý chung cư.

Như vậy, chương 4 đã hoàn thành việc mô tả toàn diện chu trình phát triển sản phẩm, từ những định hướng kiến trúc tổng quan ở tầng hệ thống cho đến các giải pháp thiết kế chi tiết đối với từng thành phần chức năng, cơ sở dữ liệu và giao diện người dùng. Thông qua các kịch bản kiểm thử nghiêm ngặt cùng mô hình triển khai thực tế được trình bày, hệ thống đã chứng minh được sự phù hợp về mặt công nghệ, tính an toàn trong bảo mật dữ liệu và hiệu năng vận hành ổn định trước các yêu cầu phi chức năng đặt ra.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tập trung trình bày các giải pháp kỹ thuật và những đóng góp cốt lõi mà đề án đã nghiên cứu, thực nghiệm và tối ưu hóa trong suốt quá trình xây dựng hệ thống quản lý chung cư. Đây là những đóng góp then chốt nhằm giải quyết triệt để các bài toán thực tế phức tạp về an ninh thông tin đa nền tảng, tự động hóa quy trình tài chính và tối ưu hóa hiệu năng chịu tải đa tầng cho toàn bộ hệ thống.

5.1 Kiến trúc xác thực lai đa nền tảng và mô hình kiểm soát quyền hạn dynamic RBAC

5.1.1 Vấn đề

Trong các hệ thống quản lý chung cư hiện đại, việc vận hành song song nhiều nền tảng ứng dụng bao gồm ứng dụng Web và ứng dụng di động là một yêu cầu bắt buộc. Sự khác biệt cốt lõi về môi trường thực thi giữa trình duyệt Web và hệ điều hành di động đặt ra một bài toán hóc búa về an ninh thông tin.

Môi trường Web đối mặt trực tiếp với các nguy cơ tấn công chèn mã độc chéo trang (XSS - Cross-Site Scripting). Nếu hệ thống lưu trữ mã báo xác thực (JWT Access Token) trong các phân vùng bộ nhớ dùng chung của trình duyệt như local-Storage hay sessionStorage, kẻ tấn công có thể dễ dàng dùng mã JavaScript độc hại để đánh cắp Token của quản trị viên tối cao, từ đó chiếm đoạt quyền điều khiển toàn bộ chuỗi tòa nhà. Môi trường Mobile App vận hành trong môi trường cô lập nên hoàn toàn miễn nhiễm với các lỗi bảo mật liên quan đến DOM hay XSS của trình duyệt. Tuy nhiên, ứng dụng di động lại yêu cầu một cơ chế lưu trữ mã báo có tính linh hoạt cao, cho phép lưu vết phiên làm việc lâu dài và hỗ trợ các luồng xử lý bất đồng bộ mà không làm đứt gãy trải nghiệm người dùng.

Nếu hệ thống áp dụng một cơ chế cấu hình xác thực đồng nhất cho cả hai môi trường, hệ thống hoặc sẽ rơi vào trạng thái mất an toàn nghiêm trọng trên Web, hoặc sẽ tạo ra rào cản kỹ thuật phức tạp gây nghẽn luồng trên thiết bị di động. Do đó, bài toán đặt ra là phải xây dựng một Kiến trúc xác thực lai có khả năng cô lập rủi ro an ninh theo từng ngữ cảnh thiết bị, kết hợp với màng lọc phân quyền động để bảo vệ an toàn tuyệt đối cho các đầu cuối API đầu vào.

5.1.2 Giải pháp

Để giải quyết triệt để bài toán trên, đề án triển khai giải pháp an ninh hai lớp. Lớp thứ nhất sử dụng chiến lược lưu trữ, vận chuyển mã báo Token phân nhánh theo thiết bị đầu cuối. Lớp thứ hai sử dụng mô hình kiểm soát quyền hạn dựa trên vai trò độc lập thông qua việc tùy biến Middleware kiểm soát của Django REST

Framework.

Hệ thống phân tách hoàn toàn cách thức tiếp nhận và xử lý JWT tùy thuộc vào định danh Client gửi yêu cầu. Với cơ chế bảo mật trên Web, sau khi tài khoản Ban quản trị xác thực thành công, máy chủ Backend đóng gói JWT Access Token vào một Cookie đặc biệt được cấu hình nghiêm ngặt với cờ `HttpOnly = True` và `Secure = True`. Với cơ chế linh hoạt trên Mobile App, hệ thống cho phép ứng dụng di động của cư dân lưu trữ Token trong phân vùng mã hóa của thiết bị và đính kèm thủ công vào tiêu đề (`Authorization: Bearer`) trong mỗi request gửi đi.

Sau khi vượt qua chốt chặn xác thực danh tính đầu vào và thiết lập được ngữ cảnh người dùng (`request.user`), hệ thống ngay lập tức đẩy request qua lớp màng lọc phân quyền thứ hai. Hệ thống tiến hành kế thừa và ghi đè lớp `BasePermission` của Django REST Framework để định nghĩa ra 4 bộ lọc quyền hạn chuyên biệt: `SuperAdmin`, `Admin`, `Householder`, `Tenant`. Các bộ lọc này hoạt động như một hệ thống tường lửa nội bộ cấu hình trực tiếp tại các lớp điều khiển xử lý (`Views`). Khi một yêu cầu can thiệp dữ liệu được gửi đến hệ thống (ví dụ: yêu cầu Tạo bảng thu phí tháng mới), lớp điều khiển sẽ khai báo nghiêm ngặt: `permission_classes = [IsAdmin]`. Nếu một tài khoản cư dân sử dụng các công cụ bên ngoài nhằm giả lập request và gọi đến API này, hệ thống sẽ lập tức kiểm tra vai trò, từ chối thực thi và trả về mã phản hồi 403 Forbidden, ngăn chặn mọi hành vi can thiệp trái phép xuống cơ sở dữ liệu.

5.1.3 Kết quả

Việc áp dụng kiến trúc xác thực lai kết hợp màng lọc phân quyền tùy biến mang lại ba ưu điểm, giải quyết các lỗ hổng tấn công.

Đầu tiên là kháng độc lập và triệt tiêu hoàn toàn nguy cơ tấn công XSS trên nền tảng Web. Ưu điểm lớn nhất của việc áp dụng cờ `HttpOnly = True` cho Cookie chứa Token trên trình duyệt Web là cô lập hoàn toàn mã báo xác thực khỏi môi trường JavaScript. Trong các hệ thống thông thường lưu Token vào `localStorage`, chỉ cần hệ thống dính một lỗ hổng chèn script nhỏ (XSS), hacker có thể lấy cắp phiên làm việc. Với giải pháp của đề tài, trình duyệt từ chối tuyệt đối không cho bất kỳ đoạn mã script nào tiếp cận Cookie này. Ngay cả khi Frontend bị tấn công chèn mã độc thành công, mã báo của Ban quản trị vẫn an toàn tuyệt đối, loại bỏ nguy cơ chiếm quyền điều khiển diện rộng. Đồng thời, cờ `Secure` ràng buộc dữ liệu chỉ truyền đi trên kênh HTTPS, ngăn chặn hoàn toàn nguy cơ nghe lén gói tin trên đường truyền.

Tiếp đến là tối ưu hóa hiệu năng kết nối và duy trì trạng thái phiên mượt mà trên Mobile App. Trái ngược với rào cản của Cookie trên trình duyệt, việc ứng dụng di động sử dụng phương thức truyền Token qua Header mang lại tính linh hoạt và

chủ động cho Mobile Client. Mobile App hoàn toàn chủ động trong việc bắt mã lỗi 401 Unauthorized từ Gateway. Hệ thống lập tức kích hoạt hàm làm mới mã báo (Refresh Token) hoàn toàn tự động dưới nền mà không cần người dùng phải đăng xuất và đăng nhập lại, giải quyết bài toán trải nghiệm người dùng liền mạch. Giúp ứng dụng dễ dàng tương tác với các dịch vụ nền của hệ điều hành di động (như background tasks, push notifications) mà không bị phụ thuộc vào cơ chế quản lý Cookie phức tạp và thiếu ổn định trên môi trường di động.

Bên cạnh đó, hệ thống được xây dựng chốt chặn an ninh đa lớp vững chắc tại tầng API dịch vụ. Mô hình phân quyền dựa trên việc tùy biến lớp BasePermission của Django REST Framework mang lại ưu điểm vượt trội về khả năng phòng thủ chủ động. Kẻ tấn công không thể sử dụng các công cụ can thiệp API (như Postman) để gửi các Request giả mạo nhằm can thiệp vào các chức năng của Ban quản lý. Mọi Request khi chạm vào lớp điều khiển (Views) đều bị ép buộc đi qua màng lọc đối soát nhãn vai trò cấu hình cứng. Ưu điểm về mặt công nghệ của giải pháp này là tính bao đóng rất cao. Khi Ban quản lý tòa nhà có nhu cầu mở rộng thêm các nhóm quyền mới, nhà phát triển chỉ cần khởi tạo thêm một lớp kế thừa BasePermission và gán vào thuộc tính permission_classes của View tương ứng. Luồng logic xác thực cốt lõi hoàn toàn không bị xáo trộn, giúp mã nguồn giữ được sự trong sáng, minh bạch và dễ dàng nâng cấp bảo trì trong tương lai.

5.2 Giải pháp thanh toán tự động với Webhook SePay

5.2.1 Vấn đề

Trong bài toán vận hành và quản lý một tổ hợp chung cư, phân hệ quản lý tài chính và thu phí luôn là trọng tâm nhưng lại đối mặt với nhiều nguy cơ sai sót. Theo phương thức vận hành truyền thống, quy trình thanh toán hóa đơn hàng tháng thường được thực hiện qua hai hình thức chính là đóng tiền mặt trực tiếp tại văn phòng Ban quản lý hoặc chuyển khoản ngân hàng tự do. Cả hai phương thức này đều bộc lộ những hạn chế lớn về mặt tối ưu nhân lực và hệ thống phần mềm.

Khi cư dân thực hiện chuyển khoản thủ công, kế toán tòa nhà phải liên tục đăng nhập vào hệ thống ngân hàng để kiểm tra sao kê biến động số dư, sau đó tiến hành dò tìm mã căn hộ dựa trên nội dung chuyển khoản để cập nhật trạng thái hóa đơn. Quy trình này tiêu tốn rất nhiều giờ làm việc mỗi ngày, đồng thời cực kỳ dễ xảy ra sai sót hoặc nhầm lẫn.

5.2.2 Giải pháp

Để hiện thực hóa quy trình thanh toán tự động khép kín, đề án đề xuất giải pháp phối hợp đa tầng bằng cách sử dụng cơ chế Webhook Gateway thông qua hệ thống SePay tại tầng máy chủ dịch vụ để tiếp nhận biến động số dư bất đồng bộ, kết hợp

giải thuật Short Polling kiểm soát vòng đời bằng React Native tại tầng ứng dụng di động để tối ưu hóa trải nghiệm giao diện người dùng.

Tại tầng Backend, hệ thống tích hợp cổng tự động hóa ngân hàng thông qua lớp xử lý chuyên biệt. Khi cư dân thực hiện luồng thanh toán hóa đơn trên thiết bị di động, hệ thống sẽ tự động cấu hình một đường dẫn mã QR động tuân thủ tiêu chuẩn VietQR, trong đó số tiền và nội dung chuyển khoản được khóa cứng bằng chuỗi định danh duy nhất chứa ID của hóa đơn. Khi cư dân quét mã và thực hiện chuyển khoản thành công, dòng tiền đi qua mạng lưới chuyển mạch Napas vào tài khoản ngân hàng của Ban quản lý, lập tức kích hoạt một tín hiệu HTTP POST chứa dữ liệu giao dịch bảo mật bắn trực tiếp về máy chủ. Tại đây, hệ thống thực hiện quy trình xác thực chữ ký số trong tiêu đề để đảm bảo gói tin đến từ cổng thanh toán hợp pháp, chống lại các cuộc tấn công giả mạo giao dịch. Sau đó, hệ thống xử lý để bóc tách chuỗi nội dung chuyển khoản nhằm tìm ra chính xác ID của hóa đơn cần xử lý, tự động khởi tạo một bản ghi giao dịch và cập nhật trạng thái của hóa đơn từ chưa thanh toán sang đã thanh toán theo nguyên tắc toàn vẹn dữ liệu.

Trong khi máy chủ xử lý bất đồng bộ qua Webhook, tại giao diện ứng dụng di động, cư dân cần được nhìn thấy trạng thái phản hồi chuyển đổi giao diện ngay lập tức mà không cần phải thực hiện thao tác tải lại trang thủ công. Hệ thống đã triển khai một cơ chế quét trạng thái chu kỳ chủ động tại màn hình thanh toán của ứng dụng React Native. Thay vì tạo kết nối liên tục gây tốn kém tài nguyên, ứng dụng di động sẽ thiết lập một bộ định thời nội bộ để gửi yêu cầu kiểm tra gọn nhẹ lên cổng kiểm tra trạng thái sau mỗi chu kỳ ngắn cố định là ba giây một lần. Khi hàm kiểm tra này nhận được tín hiệu phản hồi xác nhận hóa đơn đã được xử lý thành công từ máy chủ, ứng dụng sẽ ngay lập tức cập nhật trạng thái giao diện, kích hoạt hộp thoại thông báo thành công cho cư dân và tự động điều hướng người dùng quay trở lại danh sách hóa đơn chính.

Điểm mấu chốt trong giải pháp quản lý tài nguyên thiết bị nằm ở việc kiểm soát vòng đời của bộ định thời thông qua các hàm móc vòng đời của React Native. Hệ thống được lập trình để đảm bảo rằng ngay khi trạng thái hóa đơn chuyển sang đã thanh toán, hoặc khi cư dân chủ động thoát khỏi màn hình thanh toán để quay về trang trước, vòng lặp quét trạng thái sẽ lập tức bị triệt tiêu hoàn toàn khỏi bộ nhớ. Giải pháp này giúp loại bỏ nguy cơ rò rỉ bộ nhớ của thiết bị cầm tay, đồng thời ngăn chặn việc gửi các yêu cầu vô nghĩa lên máy chủ, làm hao phí tài nguyên RAM và dung lượng mạng của cư dân khi không còn nhu cầu sử dụng.

5.2.3 Kết quả

Sự phối hợp giữa cơ chế đẩy Webhook ở máy chủ Backend và cơ chế quét Short Polling ở ứng dụng di động đã đem lại hiệu quả vận hành cho hệ thống phần mềm quản lý chung cư, thể hiện qua các ưu điểm kỹ thuật thực tế.

Trước hết, giải pháp giúp tự động hóa hoàn chỉnh quy trình tài chính và giảm thiểu tối đa thời gian xử lý công nợ. Quy trình đối soát hóa đơn được rút ngắn từ vài giờ làm việc của kế toán xuống còn dưới hai giây kể từ thời điểm hệ thống ngân hàng ghi nhận dòng tiền thành công. Việc này triệt tiêu hoàn toàn các rủi ro sai sót, nhầm lẫn hoặc thất thoát tài chính do Ban quản lý nhập liệu thủ công, tạo nên một cơ chế minh bạch và chính xác tuyệt đối.

Tiếp theo, cơ chế này mang lại trải nghiệm giao diện mượt mà và hiện đại cho cư dân. Người dùng không cần thực hiện bất kỳ thao tác xác nhận phức tạp nào như chụp ảnh biên lai hay nhấn nút xác nhận đã chuyển tiền sau khi giao dịch. Giao diện ứng dụng di động tự động nhận biết biến động trạng thái dưới hệ thống nhờ vòng lặp quét thông minh, đưa ra phản hồi tức thì tương đương các ứng dụng ngân hàng và thương mại điện tử lớn hiện nay trên thị trường.

Cuối cùng, giải pháp chứng minh được tính tối ưu trong việc giảm áp lực chịu tải hệ thống và quản lý tài nguyên. Bằng việc thiết lập chu kỳ quét hợp lý kết hợp với cơ chế giải phóng bộ nhớ chủ động khi component đóng lại, máy chủ Backend chỉ phải chịu tải các truy vấn kiểm tra trạng thái dạng khóa đơn giản, mượt mà. Hệ thống hoàn toàn tránh được việc phải duy trì các kết nối WebSocket nặng nề và phức tạp, giúp phần mềm vận hành ổn định, không gặp hiện tượng nghẽn mạng ngay cả trong các khung giờ cao điểm thu phí của toàn bộ tòa nhà.

Dưới đây là toàn bộ nội dung mục Vấn đề đã được hiệu chỉnh lại một cách đồng bộ. Nội dung đã bóc tách phần hóa đơn ra khỏi bài toán nghẽn luồng đọc, thay vào đó xoáy sâu vào tần suất truy cập đồng thời rất lớn của cư dân và ban quản lý khi tra cứu kho tài liệu quy chế chung cư cùng danh mục tài sản thiết bị toàn tòa nhà, kết hợp với áp lực xử lý từ hai tác vụ ngầm tính toán tự động:

5.3 Giải pháp tối ưu hóa hiệu năng với Celery và Redis Caching

5.3.1 Vấn đề

Một tổ hợp chung cư đa tòa nhà quy mô lớn sở hữu mật độ thông tin dày đặc với hàng ngàn cư dân cư trú đồng thời. Trong quá trình vận hành hằng ngày, hệ thống phải đối mặt với hai thách thức cốt lõi về mặt hiệu năng chịu tải tại tầng lưu trữ và luồng xử lý.

Thứ nhất là sự bùng nổ của tần suất truy cập đồng thời từ cả phía cư dân và các

thành viên Ban quản lý khi tiến hành tra cứu kho tài liệu số hóa, quy chế nội bộ tòa nhà, hoặc kiểm tra, cập nhật danh mục thông tin tài sản thiết bị kỹ thuật trên toàn bộ các tòa nhà. Luồng dữ liệu đọc các thông tin mang tính chất cố định này nếu liên tục đổ dồn trực tiếp xuống hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL sẽ lập tức vắt kiệt tài nguyên của hệ thống lưu trữ tĩnh, gây ra hiện tượng nghẽn cổ chai tại vùng tài nguyên kết nối, làm kéo dài thời gian phản hồi API và giảm hiệu năng tổng thể của hệ thống.

Thứ hai là áp lực tính toán tài nguyên cực kỳ lớn từ các tác vụ quản trị tự động có tính chu kỳ. Điển hình như luồng nghiệp vụ quét toàn bộ hệ thống cơ sở dữ liệu để lọc ra danh sách tài sản sắp đến ngày bảo trì định kỳ nhằm gửi email cho Ban quản trị, hoặc tác vụ kiểm tra hàng ngàn căn hộ để phân loại trạng thái nợ phí nhằm đồng bộ hóa cả hai kênh truyền thông là phát thông báo trên ứng dụng di động và soạn thảo nội dung email động gửi đến từng cư dân. Nếu các tác vụ tính toán phức tạp kết hợp gửi dữ liệu ngoại vi này được thực thi tuần tự trên luồng xử lý chính của máy chủ API, hệ thống sẽ rơi vào trạng thái treo kết nối, gây sập dịch vụ cục bộ và phá vỡ sự đảm bảo về yêu cầu phi chức năng.

5.3.2 Giải pháp

Để giải quyết rào cản nghẽn tải đa tầng, đề án triển khai kiến trúc tối ưu hóa hiệu năng chịu tải đa tầng dựa trên hai chiến lược hỗ trợ là cơ chế lưu trữ đệm phân tán cho luồng đọc dữ liệu tĩnh và cơ chế phân rã, xử lý tác vụ bất đồng bộ liên nền tảng thông qua hệ sinh thái Celery kết hợp hạ tầng hàng đợi tin nhắn Redis Broker.

Hệ thống tích hợp thư viện django-redis tại tầng điều khiển để áp dụng mô hình kiến trúc bộ nhớ đệm Cache-Aside. Đối với các yêu cầu tra cứu thông tin có tần suất đọc cực cao từ phía người dùng nhưng biên độ biến đổi dữ liệu thấp bao gồm hồ sơ tài liệu quy chế tòa nhà và danh mục thông tin tài sản thiết bị chung cư, máy chủ Backend sẽ đóng vai trò trung gian điều phối dòng tiền dữ liệu. Khi nhận được yêu cầu truy vấn từ phía Client, hệ thống sẽ tiến hành kiểm tra sự tồn tại của dữ liệu định danh trong bộ nhớ RAM hiệu năng cao của Redis dưới dạng cặp khóa và giá trị. Nếu xảy ra hiện tượng trùng mục tiêu đệm, dữ liệu đã được tuần tự hóa thành chuỗi JSON sẽ lập tức được trả về cho Frontend mà không cần khởi tạo bất kỳ truy vấn SQL nào xuống đĩa cứng. Ngược lại, nếu xảy ra hiện tượng trượt đệm, hệ thống mới thực thi truy vấn xuống cơ sở dữ liệu PostgreSQL để bóc tách thông tin tài sản hoặc file tài liệu, trả kết quả cho người dùng, đồng thời tái lưu trữ dữ liệu này vào Redis đi kèm một khoảng thời gian sống giới hạn nhằm tránh lãng phí RAM. Để bảo bọc tính nhất quán thông tin, khi Ban quản lý thực hiện chỉnh sửa thông số thiết bị, cập nhật trạng thái tài sản hoặc thay đổi tệp tin tài liệu gốc, cơ

chế hủy bỏ bộ đệm lỗi thời sẽ được kích hoạt tức thì, ép buộc hệ thống xóa khóa đệm cũ và cập nhật lại dòng dữ liệu mới nhất vào bộ nhớ RAM.

Để giảm áp lực xử lý từ luồng nghiệp vụ nhắc nợ và bảo trì, hệ thống tiến hành bóc tách toàn bộ logic tính toán ra khỏi luồng xử lý phản hồi API trực tiếp, chuyển giao hoàn toàn cho phân hệ Celery Worker và Celery Beat xử lý ngầm ở tầng nền, kết nối thông qua biến môi trường URL cấu hình lõi Broker trung chuyển gói tin của Redis.

Đối với tác vụ kiểm tra và nhắc nhở lịch bảo trì tài sản, thay vì ban quản lý phải tự rà soát thủ công, hệ thống cấu hình bộ lập lịch chu kỳ Celery Beat tự động phát lệnh vào thời gian cố định mỗi ngày thông qua cấu hình lịch trình định sẵn. Lệnh này được đẩy vào hàng đợi của Redis Broker dưới dạng một thông điệp tác vụ mã hóa. Phân hệ Celery Worker đang lắng nghe trên hàng đợi sẽ tiếp nhận gói tin này, tự động tính toán mốc thời gian ngày mai và thực thi truy vấn tối ưu hóa chống lỗi truy vấn lặp bằng hàm lọc kết hợp để gom toàn bộ dữ liệu hồ sơ bảo trì. Sau đó, tiến trình tự động bóc tách danh sách email hoạt động của Admin để khởi chạy luồng gửi thư thông báo chi tiết về vị trí, chi phí dự kiến và đơn vị thực hiện một cách hoàn toàn tự động mà không chiếm dụng tài nguyên của máy chủ dịch vụ API chính.

Đối với tác vụ quét công nợ và phát thông báo nhắc phí đa kênh, tác vụ tính toán công nợ quy mô lớn được kích hoạt tự động. Tiến trình ngầm thực hiện quét lọc toàn bộ các hóa đơn chưa thanh toán có hạn nộp sắp đến trong vòng ba ngày hoặc đã rơi vào trạng thái quá hạn. Hệ thống tự động phân loại trạng thái để soạn thảo nội dung tương ứng là soạn thông điệp cảnh báo nghiêm ngặt kèm số ngày trễ nếu đã quá hạn, hoặc chèn nội dung nhắc nhở nếu sắp đến hạn. Tác vụ tiếp tục thực hiện phân rã song song luồng xử lý sang hai hướng bao gồm khởi tạo bản ghi thông báo kèm mỗi quan hệ người nhận ở trạng thái chưa đọc trên ứng dụng di động, và bóc tách tập hợp email duy nhất của cư dân để kích hoạt dịch vụ gửi mail SMTP hàng loạt thông qua cấu hình lõi gửi mail của Django.

Để đối phó với các bất ổn định về hạ tầng mạng hoặc trạng thái khóa dữ liệu tạm thời của cơ sở dữ liệu khi chạy tác vụ ngầm, cả hai hàm nghiệp vụ đều được bọc chặt chẽ bởi cơ chế tự động chạy lại nếu gặp lỗi đường truyền hoặc lỗi vận hành dữ liệu. Khi xảy ra lỗi ngoại lệ kết nối từ trình điều khiển, tương tự sự cố ngắt kết nối lớp bảo mật SSL đột ngột liên nền tảng, Celery sẽ tự động kích hoạt cơ chế phòng vệ và tái thử nghiệm thực thi tác vụ tối đa ba lần, với khoảng thời gian giãn cách cấu hình cố định sáu mươi giây, giúp hệ thống đạt độ bền bỉ và khả năng tự phục hồi lỗi vượt trội.

5.3.3 Kết quả

Việc triển khai cấu trúc hạ tầng phối hợp nhịp nhàng giữa tầng đệm tĩnh và tầng xử lý ngẫu nhiên bất đồng bộ qua Redis đã mang lại những cải thiện về mặt hiệu năng. Hệ thống được giảm tải áp lực tại tầng dữ liệu khi giảm thiểu áp lực truy vấn và số lượng phiên kết nối trực tiếp xuống hệ quản trị PostgreSQL.

Đồng thời, giải pháp giúp tối ưu hóa tốc độ phản hồi API khi tăng tốc độ phản hồi các yêu cầu tra cứu thông số tài sản và hiển thị văn bản tài liệu cho cư dân lẫn ban quản lý nhờ vào việc đọc trực tiếp dữ liệu chuỗi tuần tự hóa từ bộ nhớ RAM của Redis.

Cuối cùng, kiến trúc mới giúp giải phóng tài nguyên máy chủ chính vì các tác vụ gửi email nhắc lịch bảo trì tài sản cho Ban quản lý và phát thông báo nhắc nợ đa kênh cho hàng ngàn cư dân được thực thi mượt mà ở tầng nền với tỷ lệ thành công tuyệt đối, giải phóng hoàn toàn bộ nhớ đệm và công suất bộ vi xử lý của luồng xử lý chính, bảo bọc trải nghiệm tương tác mượt mà và không độ trễ cho người dùng trên toàn bộ hệ thống

Tóm lại, thông qua việc nghiên cứu, thiết kế và triển khai đồng bộ ba giải pháp cốt lõi bao gồm kiến trúc xác thực lai đa nền tảng, cơ chế tự động hóa tài chính qua Webhook Gateway và kiến trúc tối ưu hóa hiệu năng chịu tải đa tầng bằng cơ chế xử lý bất đồng bộ, đồ án đã hoàn thiện một hệ sinh thái công nghệ toàn diện cho tổ hợp tòa nhà chung cư. Các kết quả thực nghiệm đạt được không chỉ chứng minh tính đúng đắn về mặt lý thuyết công nghệ mà còn khẳng định khả năng ứng dụng thực tế cao, đáp ứng toàn diện các tiêu chuẩn về cả tính năng nghiệp vụ lẫn yêu cầu an toàn phi chức năng của một đô thị thông minh hiện đại.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sau quá trình nghiên cứu lý thuyết và hiện thực hóa mã nguồn, sản phẩm ứng dụng quản lý vận hành chung cư thương mại đã hoàn thành các mục tiêu cốt lõi đã đề ra. Hệ thống thể hiện được những đặc trưng ưu việt và tính ứng dụng cao. Trước hết, nền tảng đảm bảo trọn vẹn các nghiệp vụ hành chính thiết yếu như số hóa hồ sơ nhân khẩu, quản lý phương tiện, theo dõi trạng thái căn hộ và cấu hình linh hoạt các đợt thu phí dịch vụ. Điểm sáng giá của hệ thống nằm ở việc tích hợp thành công SePay API, thay thế hoàn toàn phương thức đối soát tài chính thủ công tốn kém thời gian bằng giải pháp tự động hóa toàn diện. Nhờ cơ chế này, mã VietQR được sinh động theo từng hóa đơn, cho phép hệ thống gạch nợ tự động trong thời gian thực với độ trễ dưới 3 giây, giúp giảm thiểu sai sót đáng kể cho Ban quản lý.

Về mặt kiến trúc phần mềm, hệ thống được tổ chức theo mô hình Client-Server phân tách rõ ràng. Cách tiếp cận này giúp cấp phát dữ liệu dạng JSON song song mượt mà từ máy chủ dịch vụ Backend đến cả hai nền tảng Client độc lập là React (Web) và React Native (Mobile App). Bên cạnh hiệu năng, yếu tố bảo mật cũng được ưu tiên hàng đầu; hệ thống lưu trữ Token xác thực bằng Cookie phía Web để giảm thiểu nguy cơ chiếm đoạt thẻ, đồng thời phòng chống triệt để lỗ hổng SQL Injection nhờ cơ chế tham số hóa câu lệnh mặc định của Django ORM.

Trong suốt quá trình thực hiện đồ án, tôi đã hoàn thành khối lượng công việc đáng kể từ giai đoạn khảo sát, phân tích đến thiết kế và triển khai thực tế. Tôi đã xây dựng hoàn chỉnh hệ thống API Backend bằng Django REST Framework trên nền tảng PostgreSQL, phát triển giao diện Web Admin bằng React và ứng dụng di động dành cho cư dân bằng React Native kết hợp TypeScript. Ngoài ra, việc hiện thực hóa thành công luồng xử lý bất đồng bộ bằng Celery và Redis tại môi trường phát triển cục bộ để giải quyết các tác vụ nặng như tính phí lũy tiến đồng loạt hay gửi email nhắc lịch bảo trì tự động là bước tiến quan trọng trong việc hoàn thiện kỹ năng của bản thân.

6.2 Hướng phát triển

Để đưa sản phẩm từ môi trường thử nghiệm vào áp dụng thực tế tại các tòa nhà chung cư, tôi đã xây dựng lộ trình thực hiện với các công việc ưu tiên nhằm đảm bảo hệ thống vận hành ổn định và an toàn.

Trước hết, về cấu hình triển khai và hạ tầng, tôi sẽ tập trung giải quyết triệt để rào cản chính sách kết nối mạng liên nền tảng bằng cách cấu hình và deploy thành

công phân hệ tác vụ ngầm Celery Worker và Celery Beat lên cùng một cụm hạ tầng điện toán đám mây đồng nhất với cơ sở dữ liệu PostgreSQL. Việc đồng bộ hóa môi trường này sẽ giúp loại bỏ hoàn toàn lỗi ngắt kết nối lớp bảo mật SSL đột ngột, đưa luồng xử lý bất đồng bộ tự động vào vận hành chính thức trên môi trường Production để gánh vác các tác vụ quét công nợ nặng nề của tòa nhà. Song song với đó, tôi sẽ thực hiện quy trình đóng gói nghiêm ngặt và đẩy ứng dụng di động lên các kho ứng dụng chính thức là Google Play Store và Apple App Store nhằm giúp người dùng tiếp cận, cài đặt ứng dụng thuận tiện, an toàn.

Tiếp theo, tôi sẽ tập trung kiểm thử tải cho toàn bộ hệ thống, sử dụng các công cụ kiểm thử tải chuyên dụng như Locust hoặc JMeter để giả lập kịch bản hàng ngàn cư dân cùng truy cập vào một thời điểm. Dựa trên các dữ liệu thu thập được từ quá trình giả lập, tôi sẽ tiến hành tinh chỉnh các chỉ mục trong PostgreSQL, tối ưu hóa các câu truy vấn phức tạp và thiết lập mở rộng hệ thống lưu trữ đệm với Redis cho danh mục tài sản và tài liệu. Những biện pháp này sẽ giúp giảm thiểu tối đa độ trễ, đảm bảo hệ thống phản hồi tức thì ngay cả trong điều kiện chịu tải cực lớn.

Song song với đó là kiểm thử bảo mật chuyên sâu. Tôi sẽ thực hiện rà soát tỉ mỉ toàn bộ hệ thống Interceptor của Axios để kiểm soát luồng dữ liệu, cấu hình chặt chẽ các thuộc tính quan trọng như HttpOnly và SameSite cho Cookie trong môi trường Production nhằm ngăn chặn các lỗ hổng bảo mật phổ biến. Bên cạnh đó, việc cài đặt và quản lý chứng chỉ bảo mật HTTPS sẽ được thực hiện nghiêm ngặt để bảo mật đường truyền giữa hai đầu Client-Server, từ đó triệt tiêu các nguy cơ tấn công chiếm đoạt quyền truy cập hoặc đánh cắp thông tin cá nhân của cư dân.

Cuối cùng, tôi sẽ tiếp tục thực hiện các chiến lược tối ưu hóa hiệu năng dài hạn. Duy trì việc giám sát định kỳ các chỉ số hệ thống, chủ động tái cấu trúc các phân vùng dữ liệu và tinh chỉnh tài nguyên server nhằm nâng cao trải nghiệm người dùng cuối, đảm bảo rằng mỗi tương tác của cư dân trên ứng dụng đều diễn ra mượt mà và trơn tru nhất.

TÀI LIỆU THAM KHẢO

- [1] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [2] Django Software Foundation, *Security in django*. [Online]. Available: <https://docs.djangoproject.com/en/stable/topics/security/>.
- [3] Django Software Foundation, *Django documentation and django rest framework*. [Online]. Available: <https://www.djangoproject.com/>.
- [4] Meta Platforms, *React and react router documentation*. [Online]. Available: <https://react.dev/>.
- [5] Meta Platforms, *React native and expo documentation*. [Online]. Available: <https://reactnative.dev/>.
- [6] T. Fidry, *Python-dotenv documentation*. [Online]. Available: <https://pypi.org/project/python-dotenv/>.
- [7] The PostgreSQL Global Development Group, *Postgresql documentation*. [Online]. Available: <https://www.postgresql.org/docs/>.
- [8] M. Deblois, *Axios documentation*. [Online]. Available: <https://axios-http.com/docs/intro>.
- [9] The Sass Team, *Sass documentation*. [Online]. Available: <https://sass-lang.com/documentation/>.
- [10] Redis Ltd., *Redis documentation*. [Online]. Available: <https://redis.io/docs/>.
- [11] SePay Team, *SePay api documentation*. [Online]. Available: <https://docs.sepay.vn/>.
- [12] Cloudinary Ltd., *Cloudinary documentation*. [Online]. Available: <https://cloudinary.com/documentation>.